大模型推理与训练

从零跑通

原理 · 数学 · Python/Go 单机实践

把“预测下一个字”拆到每一个数字，再亲手跑一遍

读者假设：高中数学；多年 Go 后端；不要求大模型/算法/GPU 基础

覆盖内容：Tokenizer、注意力、KV Cache、采样、训练、反向传播、分布式训练、推理工程与容量规划

配套代码：纯 Python 与纯 Go，无第三方依赖，单机可跑

配套《大模型训练与推理工程详解》的实战精读

2026 年 10 月

目录

写给读者 **vi**

第一章 从“预测下一个字”开始 **1**

1.1 一句话定义 . . . 1
1.2 一次推理的七个阶段 . . . 1
1.3 一次训练的两个循环 . . . 1
1.4 本书的玩具模型 . . . 3
1.5 本书与《工程详解》的对应 . . . 3
1.6 本章小结 . . . 4

第二章 高中数字以够用的数学 **5**

2.1 向量与矩阵：把 token 变成一个点 . . . 5
2.2 导数与梯度：往哪个方向改 . . . 6
2.3 概率：softmax 与交叉熵 . . . 7
2.4 浮点数：精度、范围与显存 . . . 8
2.5 计算图与自动微分 . . . 9
2.6 CPU、GPU 与 roofline . . . 10
2.7 复杂度与本书符号 . . . 11
2.8 本章小结 . . . 11

第三章 Tokenizer 与 Embedding：文本怎么变成向量 **12**

3.1 为什么需要 Tokenizer . . . 12
3.2 字符级 Tokenizer：本书的选择 . . . 12
3.3 BPE：把高频片段合并成 token . . . 13
3.4 Embedding：ID 到向量 . . . 14
3.5 位置编码：告诉模型“第几个” . . . 15
3.6 完整走查：从文本到矩阵 . . . 15
3.7 工程视角 . . . 16
3.8 本章小结 . . . 16

第四章 注意力：Q、K、V 到底在做什么 **17**

4.1 为什么需要上下文 . . . 17
4.2 一个 3 个 token 的手算 . . . 17
4.3 为什么要除以 $\sqrt{D}$ . . . 19
4.4 多头注意力 . . . 19
4.5 Transformer Block：注意力之外还有什么 . . . 20
4.6 本书玩具模型的完整前向 . . . 21
4.7 代码走查 . . . 21
4.8 常见误解 . . . 23
4.9 本章小结 . . . 23

第五章 从 Logits 到字符：采样 24
5.1 Logits 到概率 . . . 24
5.2 四种常见采样 . . . 24
5.3 随机数与可复现性 . . . 25
5.4 本书 demo 的真实输出 . . . 26
5.5 代码 . . . 26
5.6 工程视角 . . . 27
5.7 本章小结 . . . 27

第六章 KV Cache：把重复计算变成一次读取 28
6.1 没有缓存时，Decode 在重复算什么 . . . 28
6.2 缓存里到底存什么 . . . 29
6.3 Prefill 与 Decode 的两种账本 . . . 29
6.4 把缓存写出来 . . . 30
6.5 对拍：缓存必须和全量重算完全一致 . . . 31
6.6 显存墙：一张卡能放多少条序列 . . . 31
6.7 算术强度：为什么 batch 能救 Decode . . . 32
6.8 工程视角 . . . 32
6.9 练习 . . . 32

第七章 完整跑通：Python 与 Go 单机 demo 33
7.1 目录与两条命令 . . . 33
7.2 Python 输出逐行读 . . . 33
7.3 Go 输出与对拍 . . . 35
7.4 你可以马上做的四个实验 . . . 36
7.5 故障排查 . . . 36

7.6 从 demo 到生产：还差什么 . . . 36
7.7 本章小结 . . . 37

第八章 推理工程：demo 之外的 90% 38
8.1 批处理：从静态 barrier 到连续调度 . . . 38
8.2 PagedAttention：给 KV Cache 做虚拟内存 . . . 39
8.3 量化：让权重和 KV 变小 . . . 40
8.4 FlashAttention：别让注意力矩阵落地 . . . 40
8.5 投机解码：小模型起草，大模型验收 . . . 41
8.6 PD 分离：推理集群的读写分离 . . . 41
8.7 SLO 与容量规划 . . . 42
8.8 生产检查清单 . . . 42
8.9 本章小结 . . . 43

第九章 一次参数更新：从 2 个参数开始 44
9.1 一个只有 2 个参数的神经元 . . . 44
9.2 从标量到矩阵：公式不变 . . . 45
9.3 自动微分：让框架替你走链式法则 . . . 46
9.4 Go 版：把梯度手写出来 . . . 46
9.5 为什么需要非线性 . . . 47
9.6 从 2 个参数到 6472 个参数 . . . 47
9.7 训练循环的骨架 . . . 48
9.8 本章小结 . . . 48

第十章 语言模型训练：交叉熵与反向传播 49
10.1 数据：把文本错一位 . . . 49
10.2 损失：交叉熵 . . . 50
10.3 反向传播：逐个算子 . . . 50
10.4 优化器：AdamW . . . 52
10.5 Python 训练：真实输出 . . . 53
10.6 Go 训练：手写反向传播 . . . 53
10.7 调试：loss 不降怎么办 . . . 54
10.8 从训练到推理 . . . 54
10.9 本章小结 . . . 54

第十一章 从 6472 到 7B：算力、显存、时间 56
11.1 参数量：从结构到数字 . . . 56

11.2 算力：$6ND$ 的来历 57
11.3 显存：为什么 7B 训练要 100 GB 以上 58
11.4 时间与成本 59
11.5 Scaling Law：参数、数据、算力怎么配 59
11.6 玩具模型与 7B 的对照 59
11.7 $6ND$ 的误差来源 59
11.8 本章小结 60

第十二章 真实训练流水线：从网页到可用模型 61
12.1 全景 61
12.2 数据工程 61
12.3 Tokenizer 与 Packing 62
12.4 预训练 62
12.5 SFT：只在回答上算损失 63
12.6 对齐：RLHF 与 DPO 63
12.7 评测 64
12.8 本地 mini pipeline 64
12.9 失败 playbook 65
12.10 本章小结 65

第十三章 分布式训练：把 7B 放进集群 66
13.1 数据并行与 AllReduce 66
13.2 ZeRO：把冗余状态削掉 67
13.3 张量并行 67
13.4 流水线并行 67
13.5 3D 并行与拓扑 68
13.6 混合精度 68
13.7 激活重算 68
13.8 容错与 checkpoint 68
13.9 MFU：卡到底有几张在干活 69
13.10 7B 与 70B 的集群画像 69
13.11 本章小结 69

第十四章 从单机到生产：路线图与检查清单 71
14.1 能力地图：哪些能迁移，哪些要补 71
14.2 30/60/90 天计划 71

14.3 三个练手项目 . . . 72
14.4 自测题 . . . 73
14.5 常见误区 . . . 73
14.6 继续阅读 . . . 73
14.7 最后 . . . 74

附录 A 数学推导补充 **75**
A.1 Softmax 的 Jacobian . . . 75
A.2 交叉熵对 logits 的梯度 . . . 75
A.3 LayerNorm 反向 . . . 75
A.4 注意力反向 . . . 76
A.5 AdamW 的偏差修正 . . . 76
A.6 $6ND$ 的矩阵乘推导 . . . 76

附录 B 术语表 **77**

附录 C 代码地图与运行命令 **79**
C.1 目录 . . . 79
C.2 命令 . . . 79
C.3 预期结果 . . . 80

附录 D 练习答案（选） **81**

附录 E 参考资料 **82**

写给读者

如果你能熟练地设计一个分片、缓存、限流、可观测的后端系统，却在大模型面前感到“每个字都认识，连起来不知道它在算什么”，这本书就是为你写的。

它不是另一本“大模型概论”。它假设你读过（或正在读）《大模型训练与推理工程详解》，已经知道 Prefill/Decode、KV Cache、AllReduce、MFU 这些词，但心里仍有几个没落地的问题：

- 一个 token 从文本变成向量，再变成概率，最后变回字符，中间到底经过了哪些数？
- 注意力里的 Q、K、V 为什么能表示“看哪里”？为什么要除以 $\sqrt{d}$？
- KV Cache 到底缓存了什么？为什么它能让 Decode 从 $O(n^2)$ 变成 $O(n)$？
- 反向传播的链式法则，落到一个真正的 Transformer 上，每一步在求什么？
- 为什么 7B 模型训练要 112 GB 状态，而推理只要 14 GB？
- 单机 demo 跑通之后，距离 vLLM/DeepSpeed 的生产系统还差什么？

这本书的回答方式是：**先给一张完整地图，再把地图上的每一段路亲自走一遍**。每引入一个新概念，先给直觉，再给手算，再给可运行的代码，最后才给工程上的复杂度。全书配了一套纯 Python、纯 Go 的微型实现：一个 1 层、1 个头、24 维、6472 个参数的 Transformer。它小到可以在任何笔记本上跑，却完整包含 tokenizer、embedding、注意力、LayerNorm、MLP、softmax、采样、KV Cache、反向传播、AdamW 和 checkpoint。

原书讲了什么，本书补什么

《大模型训练与推理工程详解》共 67 页、8 章：第 1 章做转型地图；第 2 章用 2 参数神经元讲一次参数更新，然后推到 $6ND$；第 3–5 章讲数据、预训练、SFT/对齐、并行、混合精度与容错；第 6–7 章讲 Prefill/Decode、KV Cache、vLLM、PagedAttention、量化、FlashAttention、投机解码、PD 分离；第 8 章讲集群、成本与路线。它的长处是**广度和工程判断**：一个后端工程师读完，能画出训练/推理系统的全貌，知道每个部件的工程动机。

它的不足，也正是本书的起点：

- **推理跳得太快**。原书从“一次只能走一步”直接进入算术强度、TTFT/TPOT、vLLM，缺少 tokenizer → embedding → QKV → logits → 采样 → detokenize 的逐段数值。读者知道“Prefill 是计算密集”，但没亲手算过一个 3 token 的注意力。
- **训练缺少可运行的反向传播**。原书从 2 参数神经元一步跳到 $6ND$ 和 7B 账本，中间“6472 个参数的梯度怎么走”是断的；读者知道链式法则，但没看过 LayerNorm/注意力的梯度代码。
- **没有代码，无法自验**。所有结论都是“作者算给你看”，读者不能改一个温度、删一行缓存、跑一遍对拍来验证自己是否真的理解。

维度	原书	本书
定位	工程广度与架构判断	从零手算 + 单机跑通
推理	直接讲系统与优化	逐段数值、完整请求、KV 对拍
训练	玩具神经元 $\to 6ND$	2 参数 $\to$ 6472 参数 $\to$ 7B
代码	无	Python + Go，零依赖，可跑
读者动作	读、记、画架构图	算、跑、改、对拍

表 1: *

两本书的分工：先读原书建立地图，再用本书把每一段路走一遍；反过来也成立。

三条主线

- **一条请求主线**。从 HTTP 请求到流式输出，沿着“tokenizer $\to$ embedding $\to$ 注意力 $\to$ logits $\to$ 采样 $\to$ detokenize”走一遍。每一站都给出真实的数字和代码。
- **一条训练主线**。从 2 个参数的神经元，到一个 6472 参数的 Transformer，再到 7B 的算力/显存账本。每一层都给出前向、损失、反向、更新的公式与手算。
- **一条工程主线**。每一章最后回答“这件事放到 7B、70B、上千张卡上，会变成什么问题”。单机 demo 是显微镜，工程章节是望远镜。

怎么读

- **第一遍（建立广度）**：读每章开头的“一句话”和每节末尾的“工程视角”，跳过代码和推导。你会在两三个小时里得到一张完整地图。
- **第二遍（建立深度）**：跟着“手算”盒子把数字算一遍，再运行对应的 Python/Go 代码。遇到公式不要跳，书里的推导只用到高中求导和矩阵乘法。
- **第三遍（建立工程感）**：读第 7、8、12、13 章，把 demo 的实测数字代入 7B 的账本，自己推一遍容量规划。

代码与约定

所有代码在 `llmbook/` 下：Python 在 `py/`，Go 在 `go/`，语料在 `data/`，权重与运行日志在 `weights/`、`out/`。Python 只用标准库，Go 只用标准库，不需要安装 PyTorch、CUDA 或任何科学计算包。这样做的代价是速度慢，好处是**没有黑盒**：每一个乘法、每一次 softmax、每一条梯度都在你能读到的代码里。

书中公式约定：向量默认是列向量，但代码里为了直观按行存放；矩阵乘法 AB 表示 A 的每一行与 B 的每一列做点积；$\odot$ 表示逐元素乘；$[T, D]$ 表示 T 行、D 列。FLOPs 是浮点运算次数，FLOPS 是每秒浮点运算次数，二者只差一个时间单位。

与《工程详解》的关系

《工程详解》回答“这个系统由哪些部件组成、为什么这样设计”；本书回答“每个部件内部的数是怎么流动的、我能不能自己写一遍”。两本书的章节对应关系见第 1 章表 1.4。建议先读《工程详解》建立术语，再读本书落地；也可以反过来，用本书的 demo 建立直觉，再回到《工程详解》看工程全貌。

写于 2026 年 10 月

第 1 章

从“预测下一个字”开始

一句话

大语言模型只做一件事：给定前面的 token，输出下一个 token 的概率分布。推理是反复做这件事，训练是调整参数让这件事做得更准。

1.1 一句话定义

设一段文本被切成 T 个 token：$x_1, x_2, \ldots, x_T$。语言模型定义条件概率

$$P_\theta(x_t \mid x_1, \ldots, x_{t-1}), \qquad t = 1, \ldots, T, \tag{1.1}$$

其中 θ 是模型参数。给定前文，模型输出词表 V 上的一个分布；下一个 token 就是从分布里采样（或取最大）得到的。生成一段长度为 N 的回答，就是把“输出一个 token、拼回前文、再输出下一个”重复 N 次。

这句话看起来简单，却解释了后面所有工程现象：

- 为什么生成是串行的：第 t 个输出依赖第 $t-1$ 个输出，不能提前算。
- 为什么 Prefill 可以并行：prompt 的所有 token 已知，可以一次算完。
- 为什么 KV Cache 有用：前面 token 的 K、V 不随新 token 改变，可以缓存。
- 为什么训练可以并行：训练时每个位置的“正确答案”都在语料里，整条序列可以同时算损失。

1.2 一次推理的七个阶段

表 1.1 是本书反复使用的地图。先不要纠结每个词，后面每一章会拆开其中一段。

1.3 一次训练的两个循环

训练不是“把知识灌进去”，而是反复做下面四步：

1. **前向**：把一批 token 喂给模型，得到每个位置的预测分布。
2. **损失**：用交叉熵衡量预测分布与真实下一个 token 的差距，得到一个标量 L。
3. **反向**：对 L 求每个参数的偏导，得到梯度 $g = \nabla_\theta L$。

步骤	名称	输入 → 输出	主要在算什么
1	接入/鉴权/限流	HTTP 请求 → 合法请求	CPU：配额、最大长度、参数校验
2	Tokenization	文本 → token ID	查表/合并规则
3	Embedding	ID → 向量	查表，$[T] \to [T, D]$
4	Prefill	prompt 向量 → 首 token + KV	矩阵乘、注意力，计算密集
5	Decode	上一步 token → 下一个分布	矩阵-向量乘，访存密集
6	采样	分布 → 一个 token	softmax、temperature、top-p
7	Detokenize/流式输出	token → 文本	查表、SSE/WebSocket

表 1.1: 一次推理请求的七个阶段。第 1、2、7 步在 CPU/网关上，第 4、5 步在 GPU 上。

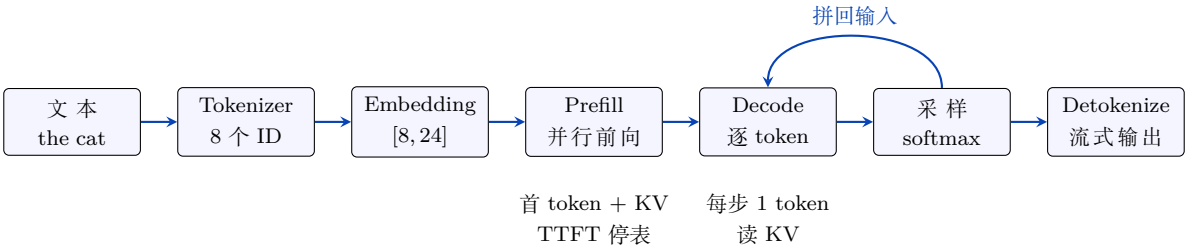


图 1.1: 一次推理请求的完整链路。Prefill 一次算完 prompt；Decode 循环“采样 → 拼回输入 → 再前向”，直到 EOS 或长度上限。

维度	训练	推理
目标	调整参数，让 loss 下降	固定参数，生成 token
状态	权重 + 梯度 + 优化器 + 激活	权重 + KV Cache
并行	数据/张量/流水线	不同请求可并行；同一请求的 token 串行
瓶颈	算力 + 集合通信	Prefill 算力；Decode 显存带宽
核心指标	MFU、总时长、loss	TTFT、TPOT、goodput

表 1.2: 训练与推理的对照。两者都叫“前向”，但资源约束和优化目标完全不同。

4. **更新**：$\theta \leftarrow \theta - \eta g$（或 AdamW 的变体），让 L 小一点。

这四步组成一个 iteration。大模型训练就是把这个 iteration 重复几十万到几百万次。第 2 章会把每一步拆到标量；第 9–10 章会把它落到 6472 个参数上。

1.4 本书的玩具模型

为了让所有数字可以手算、所有代码可以在一台笔记本上跑完，本书使用表 1.3 的模型。它不是 GPT 的缩小版“近似”，而是结构上完整的 decoder-only Transformer：token embedding、位置 embedding、单头因果自注意力、LayerNorm、ReLU MLP、残差、输出投影，一样不少。

配置	值	说明
词表大小 V	16	字符级：15 个字母/空格/句点
上下文长度 T	32	位置 embedding 的行数
层数 L	1	一个 Transformer block
注意力头数 H	1	单头，便于手算
隐藏维度 D	24	每个 token 的向量长度
FFN 维度 D_{ff}	48	先升维再降维
参数量	6472	见第 10 章逐项相加

表 1.3: 本书的微型 Transformer。代码在 `py/model.py` 与 `go/tiny/`。

语料是一段 644 个字符的英文小故事，只有 16 种字符。训练 600 步（Python）或 1200 步（Go）后，模型能续写出 `the cat ran to the log .` 这样的句子。它当然不“智能”，但它把大模型的每一个机械环节都暴露出来了。

1.5 本书与《工程详解》的对应

《工程详解》	本书
第 1 章从分布式后台到 AI Infra	第 1、2 章：建立最小数学与系统直觉
第 2 章训练的原理	第 9、10 章：从 2 个参数到 6472 个参数，手算 + 代码
第 3 章训练全流程	第 11、12 章：数据、SFT、DPO、checkpoint
第 4 章分布式并行	第 13 章：DP/TP/PP/ZeRO 的账本与 demo 之外的工程
第 5 章训练工程	第 10、13 章：混合精度、梯度裁剪、MFU、故障
第 6 章推理原理	第 3–6 章：tokenizer、注意力、采样、KV Cache
第 7 章推理工程	第 7、8 章：完整 demo、batching、PagedAttention、PD
第 8 章集群与成本	第 8、11、13 章：容量规划、算力/显存账本

表 1.4: 两本书的章节映射。本书不重复《工程详解》的架构讨论，而是补齐“数怎么流、代码怎么写”。

1.6 本章小结

- 大模型 = 条件概率 $P_\theta(x_t \mid x_{<t})$ 的参数化函数。
- 推理 = Prefill（并行算 prompt）+ Decode（串行逐 token）。
- 训练 = 前向 → 损失 → 反向 → 更新，重复。
- 本书用一个 6472 参数的 Transformer 把上述过程全部跑通。

第 2 章

高中数字以够用的数学

一句话

这一章只补五样东西：向量/矩阵、导数/链式法则、概率/softmax/交叉熵、浮点数、计算图。每一件都配一个手算例子；不需要微积分课，只需要会乘加和求导。

很多大模型教程一上来就写 $\mathrm{softmax}(QK^\top/\sqrt{d})V$，读者卡住的地方往往不是 Transformer，而是这些符号背后的“数长什么样”。本章把符号翻译成可以手算的数字。已经熟悉的读者可以快速扫过，但建议至少做一遍 2.1、2.2、2.3 的三个手算，后面所有章节都会复用它们。

2.1 向量与矩阵：把 token 变成一个点

2.1.1 向量

一个 D 维向量就是 D 个数排成一列：

$$\mathbf{x} = (x_1, x_2, \ldots, x_D)^\top \in \mathbb{R}^D. \tag{2.1}$$

在本书的玩具模型里，$D = 24$，所以每个 token 是一个 24 维向量。你可以把它想成 24 个浮点数组成的“特征表”：第 1 个数可能表示“是不是名词”，第 2 个数表示“是不是过去式”……真实模型不会给每一维起名字，但这些维度确实在训练中学会了区分 token。

两个向量的**点积**（dot product）是

$$\mathbf{a} \cdot \mathbf{b} = \sum_{i=1}^{D} a_i b_i. \tag{2.2}$$

它衡量两个向量“方向有多一致”：同向为正且大，垂直为 0，反向为负。注意力里的“相关性”就是用点积算的。

手算

取 $\mathbf{a} = (1, 2, 0)$，$\mathbf{b} = (2, 0, 1)$：

$$\mathbf{a} \cdot \mathbf{b} = 1 \times 2 + 2 \times 0 + 0 \times 1 = 2.$$

如果 $\mathbf{b} = (2, 4, 0)$，点积是 $1 \times 2 + 2 \times 4 + 0 = 10$，因为 $\mathbf{b} = 2\mathbf{a}$，方向完全一致。

2.1.2 矩阵与矩阵乘法

一个 $n \times m$ 的矩阵是 n 行 m 列的数表。矩阵乘法 $C = AB$ 要求 A 的列数等于 B 的行数：若 $A \in \mathbb{R}^{n\times m}$，$B \in \mathbb{R}^{m\times p}$，则 $C \in \mathbb{R}^{n\times p}$，且

$$C_{ij} = \sum_{k=1}^{m} A_{ik} B_{kj}. \tag{2.3}$$

换句话说，C 的第 i 行第 j 列，是 A 的第 i 行与 B 的第 j 列的点积。

手算

$$A = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}, \quad B = \begin{pmatrix} 5 & 6 \\ 7 & 8 \end{pmatrix}.$$

$$AB = \begin{pmatrix} 1\cdot 5 + 2\cdot 7 & 1\cdot 6 + 2\cdot 8 \\ 3\cdot 5 + 4\cdot 7 & 3\cdot 6 + 4\cdot 8 \end{pmatrix} = \begin{pmatrix} 19 & 22 \\ 43 & 50 \end{pmatrix}.$$

一次 2×2 乘 2×2 需要 $2\times 2\times 2 = 8$ 次乘法、8 次加法，共 16 次浮点运算（FLOPs）。

一般地，$n \times m$ 乘 $m \times p$ 的 FLOPs 约为 $2nmp$（每个输出元素 m 次乘加，乘加各算 1 次）。这个“2”会一直用到第 11 章的 $6ND$ 公式。

2.1.3 从向量到批量：形状

深度学习的代码里，最重要的信息往往不是数值，而是**形状**（shape）。本书用 $[T, D]$ 表示 T 行、D 列；$[B, T, D]$ 表示 batch 维 B。一次前向里，形状会这样变化：

阶段	形状	说明
token ID	$[T]$	T 个整数
embedding	$[T, D]$	每个 ID 查一行
Q/K/V	$[T, D]$	三个线性投影
注意力分数	$[T, T]$	每个位置对每个位置
注意力输出	$[T, D]$	加权求和
logits	$[T, V]$	每个位置对词表的分布

把形状写对，模型就写对了一半；形状错了，报错往往比数值错误更容易发现。

2.2 导数与梯度：往哪个方向改

2.2.1 导数就是斜率

函数 $f(x)$ 在 x 处的导数 $f'(x)$ 是“x 增加一点点时，f 增加多少”的极限：

$$f'(x) = \lim_{h\to 0} \frac{f(x+h) - f(x)}{h}. \tag{2.4}$$

常见规则：

$$(x^n)' = nx^{n-1}, \qquad (e^x)' = e^x, \qquad (\ln x)' = 1/x, \tag{2.5}$$

$$(f+g)' = f' + g', \qquad (fg)' = f'g + fg', \tag{2.6}$$

$$(f(g(x)))' = f'(g(x))\,g'(x) \quad \text{（链式法则）}. \tag{2.7}$$

最后一条是反向传播的全部秘密：复合函数求导，等于外层导数乘以内层导数。

2.2.2 偏导数与梯度

一个函数有多个输入时，对第 i 个输入求导，把其他输入当常数，叫偏导数 $\partial f/\partial x_i$。把所有偏导数排成向量，就是梯度：

$$\nabla_{\mathbf{x}} f = \left(\frac{\partial f}{\partial x_1}, \ldots, \frac{\partial f}{\partial x_D} \right)^{\top}. \tag{2.8}$$

梯度指向函数上升最快的方向；要最小化 f，就往负梯度方向走。

手算

$f(w,b) = (wx + b - y)^2$，取 $x = 2, y = 3, w = 0.5, b = 0$。先算 $wx + b = 1$，再算：

$$\frac{\partial f}{\partial w} = 2(wx + b - y) \cdot x = 2(1-3) \cdot 2 = -8,$$

$$\frac{\partial f}{\partial b} = 2(wx + b - y) \cdot 1 = -4.$$

负号表示：w 增大时 f 减小。所以更新 $w \leftarrow w - \eta(-8) = w + 8\eta$。

2.2.3 数值梯度检查

如果你不确定自己写的反向传播对不对，最笨也最有效的办法是**数值梯度**：把参数加一个很小的 h，算损失；再减 h，算损失；用

$$\frac{\partial f}{\partial \theta} \approx \frac{f(\theta + h) - f(\theta - h)}{2h} \tag{2.9}$$

近似。它慢，但独立于你的反向传播代码，是最好的“对拍”工具。第 9 章的 Go 代码就带了一个这样的测试。

2.3 概率：softmax 与交叉熵

2.3.1 从分数到概率

模型最后一层输出 V 个实数，叫 logits：$z_1, \ldots, z_V$。它们不是概率，可能为负、可能大于 1。softmax 把它们变成概率：

$$p_i = \frac{e^{z_i}}{\sum_{j=1}^{V} e^{z_j}}. \tag{2.10}$$

性质：$p_i > 0$，$\sum_i p_i = 1$；z_i 越大，p_i 越大。实现时要先减去最大值，避免 e^z 溢出：

$$p_i = \frac{e^{z_i - z_{\max}}}{\sum_j e^{z_j - z_{\max}}}. \tag{2.11}$$

数学上完全等价，数值上安全得多。

手算

$z = (2, 1, 0)$。$e^2 = 7.389$，$e^1 = 2.718$，$e^0 = 1$，和 $= 11.107$。所以

$$p = (0.665, 0.245, 0.090).$$

最大 logit 的 token 有 66.5% 的概率；但仍有 33.5% 的概率选到别的 token。

2.3.2 交叉熵：给“猜得有多差”打分

真实下一个 token 是 y（词表里的一个下标）。模型给它的概率是 p_y。交叉熵损失定义为

$$L = -\ln p_y. \tag{2.12}$$

如果 $p_y = 1$，$L = 0$；如果 $p_y = 0.01$，$L = \ln 100 \approx 4.605$。模型越肯定错误答案，惩罚越大。

对一批 T 个位置取平均：

$$L = -\frac{1}{T}\sum_{t=1}^{T} \ln p_{t,y_t}. \tag{2.13}$$

困惑度（perplexity）是 e^L，可以理解为“模型在每一步平均在多少个 token 里犹豫”。$L = \ln V$ 时，困惑度是 V，相当于随机猜。

手算

词表 $V = 16$，随机猜时 $p_y = 1/16$，$L = \ln 16 = 2.773$。本书模型训练开始时 loss 约 3.58（比随机差，因为初始化不好），训练 600 步后降到 0.208，困惑度 $e^{0.208} \approx 1.23$。

2.3.3 为什么大模型用交叉熵而不是 MSE

下一个 token 是“16 选 1”或“10 万选 1”的分类问题。交叉熵在“正确 token 的概率被压得很低”时给出很大的梯度，纠错信号强；MSE 对概率的惩罚是二次的，而且 softmax 之后梯度会变小。第 10 章会推导 softmax + 交叉熵的梯度，结果非常干净：

$$\frac{\partial L}{\partial z_i} = p_i - \mathbf{1}[i = y]. \tag{2.14}$$

预测概率与 one-hot 标签的差，就是 logits 的梯度。这是全书最值得记住的公式之一。

2.4 浮点数：精度、范围与显存

2.4.1 三个部分

一个浮点数由符号、指数、尾数组成：

$$(-1)^s \times 1.m \times 2^{e - \text{bias}}. \tag{2.15}$$

指数决定“能表示多大/多小”，尾数决定“有多精细”。常见格式：

格式	总位数	符号/指数/尾数	大致范围	精度
fp32	32	1/8/23	$10^{\pm 38}$	约 7 位十进制
fp16	16	1/5/10	$10^{\pm 5}$	约 3 位
bf16	16	1/8/7	$10^{\pm 38}$	约 2–3 位
int8	8	整数	$-128 \sim 127$	步长由 scale 决定

bf16 牺牲尾数换范围，训练时不容易溢出，所以大模型训练常用 bf16；fp16 精度更高但范围小，需要 loss scaling。

2.4.2 对称量化

把浮点数组映射到 int8：

$$s = \frac{\max_i |x_i|}{127}, \qquad q_i = \mathrm{round}(x_i/s), \qquad \hat{x}_i = q_i s. \tag{2.16}$$

误差上界是 $s/2$。s 可以整个张量共用一个（per-tensor），也可以每行一个（per-channel）；后者误差更小，但要多存一些 scale。

手算

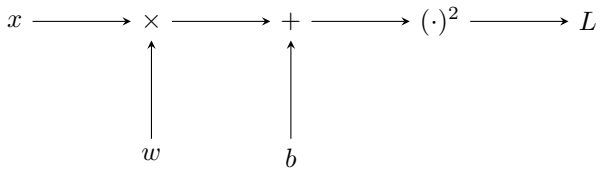
权重范围 $[-3.2, 3.2]$，$s = 3.2/127 \approx 0.0252$。$x = 1.0$ 时 $q = \mathrm{round}(1/0.0252) = 40$，反量化 $\hat{x} = 40 \times 0.0252 = 1.008$，误差 0.008，小于 $s/2 = 0.0126$。

第 8 章会用本书的 6472 参数模型跑一遍 int8 量化，观察 logits 变化和 top-1 是否保持。

2.5 计算图与自动微分

2.5.1 把计算写成图

前向传播的每一步都可以看成一个节点：输入、乘法、加法、平方、损失。节点之间有依赖，形成一张有向无环图（DAG）。反向传播就是从损失节点出发，沿着边反向走，把“上游梯度”乘以“本地导数”，累加到每个输入上。



2.5.2 一个 80 行的自动微分

PyTorch 的 `loss.backward()` 做的就是这张图的反向遍历。本书的 Python 版实现了一个极小的 autograd（`py/autograd.py`）：每个 `Tensor` 记录自己的数据和产生它的操作；`backward()` 先做拓扑排序，再倒序调用每个操作的 `\_backward`。

Listing 2.1: 自动微分引擎的核心结构（节选）

```
class Tensor:
    def __init__(self, data, shape=None, requires_grad=False, _children=()):
        self.data = [float(x) for x in data]
        self.shape = tuple(shape)
        self.grad = [0.0] * len(self.data) if requires_grad else None
        self._backward = lambda: None
        self._prev = _children

def backward(loss):
    topo, seen = [], set()
    def build(t):
        if id(t) in seen: return
        seen.add(id(t))
        for p in t._prev: build(p)
        topo.append(t)
    build(loss)
    loss.grad = [1.0]
    for t in reversed(topo):
        t._backward()
```

每个操作在构造时定义自己的反向规则。例如乘法 $c = ab$：

```
def mul(a, b):
    out = Tensor([x*y for x,y in zip(a.data,b.data)], a.shape, (a,b), True)
    def backward():
        for i,g in enumerate(out.grad):
            a.grad[i] += g * b.data[i]
            b.grad[i] += g * a.data[i]
    out._backward = backward
    return out
```

这就是链式法则的代码版：上游梯度 g 乘本地导数 b 给 a，乘 a 给 b。

2.6 CPU、GPU 与 roofline

2.6.1 为什么 GPU 快

CPU 有几十个复杂的核，擅长分支、缓存、串行逻辑；GPU 有成千上万个简单的核，采用 SIMT（单指令多线程）：同一时刻，大量线程执行同一条指令、处理不同数据。神经网络的核心是矩阵乘：同一个乘加操作施加到海量元素上，正好适合 SIMT。

但 GPU 快不只因为核多，还因为**显存带宽**。H100 的 HBM 带宽约 3 TB/s，CPU 的 DDR 约 0.3 TB/s，差 10 倍。大模型 Decode 阶段每生成一个 token 要把全部权重读一遍，瓶颈常常是“搬数据”而不是“算数据”。

2.6.2 算术强度

算术强度定义为每读 1 字节做多少次浮点运算：

$$I = \frac{\text{FLOPs}}{\text{Bytes}}. \tag{2.17}$$

硬件的“拐点”是峰值算力除以峰值带宽：

$$I^* = \frac{\text{FLOPS}}{\text{Bytes/s}}. \tag{2.18}$$

H100 级 GPU 的 I^* 约 $10^{15}/(3\times10^{12}) \approx 333$ FLOP/B。$I > I^*$ 是计算密集（卡在算力），$I < I^*$ 是访存密集（卡在带宽）。

手算

7B 模型 bf16 权重 14 GB。Prefill 1000 个 token，计算量约 $2N\times1000 = 1.4\times10^{13}$ FLOPs，访存约 14 GB，$I \approx 1000$，远大于 333，计算密集。Decode 1 个 token，计算量 $2N = 1.4\times10^{10}$，访存 14 GB，$I \approx 1$，远小于 333，访存密集。同一个模型，两个阶段的瓶颈完全不同。

2.7 复杂度与本书符号

本书用 $O(\cdot)$ 表示“随规模增长的量级”，忽略常数。例如注意力分数矩阵是 $T\times T$，所以注意力计算量是 $O(T^2D)$，KV Cache 是 $O(TD)$。但工程上常数很重要：$T = 32$ 时 T^2 只有 1024，$T = 128000$ 时是 1.6×10^{10}，完全不同的世界。

符号	含义
T	序列长度（token 数）
B	batch 大小
D	隐藏维度
V	词表大小
L	Transformer 层数
N	参数量
FLOPs	浮点运算次数（乘法和加法各算 1 次）
FLOPS	每秒浮点运算次数
B	字节（Byte）；注意与 batch 的 B 区分

2.8 本章小结

- 向量点积衡量方向一致性；矩阵乘法是“行与列的点积”，FLOPs 约 $2nmp$。
- 导数衡量“输入变一点，输出变多少”；链式法则是反向传播的全部。
- softmax 把 logits 变成概率；交叉熵是 $-\ln p_y$；softmax+CE 的梯度是 $p -$ one-hot。
- bf16 用尾数换范围；int8 用 scale 把浮点映射到整数。
- 计算图把前向写成 DAG，反向就是倒序链式法则。
- 算术强度 I 判断计算密集还是访存密集；这是理解 Prefill/Decode 的钥匙。

第 3 章

Tokenizer 与 Embedding：文本怎么变成向量

一句话

模型不会读字。Tokenizer 把文本切成整数 ID，Embedding 把每个 ID 查成一行向量；位置编码再告诉模型“这个向量排第几”。这三步之后，文本才变成 Transformer 能吃的矩阵。

3.1 为什么需要 Tokenizer

计算机只能处理数字。最直接的办法是把每个字符映射成一个整数（字符级），或者把每个单词映射成一个整数（词级）。两种做法都有问题：字符级序列太长，词级词表爆炸且遇到生词就失效。工业界的主流是子词（subword）分词：高频词整体成一个 token，低频词拆成几个片段。BPE（Byte Pair Encoding）是最常用的一种。

Token 是计费、算力、上下文长度的基本单位。同一个句子，tokenizer 不同，token 数可能差 2–3 倍；中文尤其明显：拿英文语料训的词表切中文，一个汉字可能被拆成两三个 token，成本凭空翻倍。

3.2 字符级 Tokenizer：本书的选择

为了让所有数字可以手算，本书的 demo 用字符级 tokenizer：语料的 16 种字符就是词表。

Listing 3.1: 字符级词表与编码（py/train.py 节选）

```
text = open(path).read().replace("\n", " ")
vocab = sorted(set(text))          # 16 chars
stoi = {c: i for i, c in enumerate(vocab)}
ids = [stoi[c] for c in text]      # text -> [T]
```

本书语料的词表是：

ID	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
char		.	a	c	d	e	g	h	l	m	n	o	r	s	t	w

提示词 "the cat " 被编码成：

$$\text{the cat } \rightarrow [14, 7, 5, 0, 3, 2, 14, 0].$$

这 8 个整数就是模型看到的全部输入。它不知道“the”是一个词，只知道第 14、7、5 号 token 依次出现。

字符级的优点是词表小、没有生词、代码简单；缺点是序列长、语义粒度细。真实模型用 3 万到 15 万词表的 BPE，把常见词压成一个 token，从而在同样上下文里塞进更多内容。

3.3 BPE：把高频片段合并成 token

3.3.1 算法

BPE 的训练过程只有一句话：反复统计相邻符号对的频率，把频率最高的一对合并成一个新符号。初始词表是单个字符；跑几万轮后，词表膨胀到几万项，常见词整体成 token，低频词保留片段。

手算

语料只有三个词：`cat`（3 次）、`eat`（2 次）、`fish`（2 次）。初始：

$$\mathrm{c\ a\ t} \times 3, \quad \mathrm{e\ a\ t} \times 2, \quad \mathrm{f\ i\ s\ h} \times 2.$$

相邻对频率：$(a,t)=5$，$(c,a)=3$，$(e,a)=2$，$(f,i)=2$，$(i,s)=2$，$(s,h)=2$。最高是 (a,t)，合并成 `at`：

$$\mathrm{c\ at} \times 3, \quad \mathrm{e\ at} \times 2, \quad \mathrm{f\ i\ s\ h} \times 2.$$

下一轮最高是 $(c,at)=3$，合并成 `cat`；再下一轮合并 (i,s)、(is,h)，得到 `fish`。最终词表里有 `cat`、`at`、`ish` 等符号。

本书的 `py/bpe.py` 在 644 字符的语料上跑了 24 轮，学到的前 15 个合并是：

轮次	合并	新符号	频次
1	t + h	th	40
2	th + e	the	40
3	a + t	at	40
4	o + g	og	28
5	c + at	cat	20
6	d + og	dog	20
7	s + at	sat	12
8	o + n	on	12
9	m + at	mat	8
10	l + og	log	8
11	r + a	ra	8
12	ra + n	ran	8
13	t + o	to	8
14	s + a	sa	8
15	sa + w	saw	8

最终词表从 16 个字符扩展到 30 个符号。用这套合并表切句子：

the cat sat on the mat .

得到 13 个 token：

$$[\text{the}], [\ ], [\text{cat}], [\ ], [\text{sat}], [\ ], [\text{on}], [\ ], [\text{the}], [\ ], [\text{mat}], [\ ], [.]$$

24 个字符压成 13 个 token，几乎减半。真实 BPE 词表更大，压缩率更高。

3.3.2 字节级 BPE

真实模型通常从字节（256 个可能值）开始，而不是从 Unicode 字符开始。这样任何文本（包括 emoji、罕见汉字、二进制）都能被表示，不会出现“未知 token”。代价是中文常见字可能占 1–2 个 token，词表设计需要覆盖目标语言。

3.3.3 工程坑

常见坑

- **词表与训练数据同分布**。用英文语料训的词表切中文，成本可能翻倍。词表要在目标语言上训练或扩词表。
- **特殊 token**。 `<s>`、`</s>`、`<pad>`、`<|user|>` 等要显式加入，并保证训练和推理一致。
- **版本锁定**。 tokenizer 的合并表、规范化规则（Unicode NFC/NFKC）、特殊 token 配置必须和权重一起版本化；换 tokenizer 等于换模型。
- **不要用字符数估算成本**。用 tokenizer 实际切一遍；中英文、代码、JSON 的 token/字符比差别很大。

3.4 Embedding：ID 到向量

3.4.1 查表

Embedding 是一个 $V \times D$ 的矩阵 E。第 i 个 token 的向量就是 E 的第 i 行：

$$\mathbf{x}_t = E[\mathrm{id}_t] \in \mathbb{R}^D. \tag{3.1}$$

它没有乘法，只是一次内存查表。本书 $V = 16, D = 24$，所以 E 有 $16 \times 24 = 384$ 个参数。

手算

假设第 14 号 token（`t`）的 embedding 前 6 维是

$$[0.12, -0.03, 0.44, 0.01, -0.20, 0.07, \dots].$$

这 24 个数没有人类可读的含义；它们是训练出来的。模型后续的 Q/K/V 投影会把这些数组合成“这个词在当前位置的查询/键/值”。

3.4.2 为什么需要维度

一维只能表示一个标量，无法区分“名词/动词/过去式/单复数/语义类别”。D 越大，模型能容纳的区别越多，但参数量和计算量也越大。7B 模型的 D 通常是 4096，embedding 矩阵 $V \times D$ 若有 10 万词表就是 4.1×10^8 个参数，占 7B 的约 6%。

3.4.3 权重绑定

很多模型让输入 embedding 和输出投影共享同一个矩阵（weight tying）：

$$\text{logits} = \mathbf{h}E^\top. \tag{3.2}$$

这样输出层不需要额外的 $D \times V$ 参数，小模型尤其常见。本书为了看清两个矩阵，分别使用了 `tok_emb` 和 `lm_head`。

3.5 位置编码：告诉模型“第几个”

3.5.1 为什么需要位置

如果只把 token 的 embedding 相加，模型看到的是“一个袋子里有哪些词”，不知道顺序。“猫追狗”和“狗追猫”会得到相同的集合表示。位置编码给每个位置一个向量，加到 token 向量上：

$$\mathbf{x}_t = E[\text{id}_t] + P[t]. \tag{3.3}$$

本书的 P 是一个 32×24 的可学习矩阵，每个位置一行。

手算

位置 0 的向量 $P[0]$ 和位置 1 的 $P[1]$ 不同；模型通过这个差异知道“谁在前”。如果两个 token 的 embedding 相同，它们在不同位置仍然会得到不同的 $\mathbf{x}_t$。

3.5.2 从可学习位置到 RoPE

可学习位置编码的缺点是不能外推到训练时没见过的长度。现代模型常用 RoPE（旋转位置编码）：把 Q、K 的每两维看成复数，按位置旋转一个角度。位置 t 的旋转让 $q_t \cdot k_s$ 只依赖相对距离 $t - s$，从而更好地外推。本书的 demo 仍用可学习位置，因为它的形状和加法最直观；RoPE 的细节属于“工程优化”，不影响对推理流程的理解。

3.6 完整走查：从文本到矩阵

以 `"the cat "` 为例，表 3.1 是每一步的真实形状和数值（前 3 维）。

在代码里，这一步只有三行：

```
x = embedding(params["tok_emb"], tokens)
pos = embedding(params["pos_emb"], list(range(len(tokens))))
x = add(x, pos)          # [T,D] + [T,D]
```

阶段	形状	内容（示意）
文本	–	`"the cat "`
token ID	[8]	14, 7, 5, 0, 3, 2, 14, 0
token emb	$[8, 24]$	每行 24 个数
位置 ID	[8]	0,1,2,3,4,5,6,7
位置 emb	$[8, 24]$	每行 24 个数
相加	$[8, 24]$	$\mathbf{x}_t = E[\mathrm{id}_t] + P[t]$

表 3.1: 文本到向量。之后这个 $[8, 24]$ 矩阵进入 Transformer block。

3.7 工程视角

工程视角

- **tokenizer 是服务的一部分**。网关要做 token 计数、最大长度校验、计费；tokenizer 的 CPU 开销虽小，但在高 QPS 下也会成为瓶颈，通常独立部署或放进推理前端。
- **词表大小影响显存**。$V \times D$ 的 embedding 和输出层是显存账本的一项；词表翻倍，这两块也翻倍。
- **位置长度决定上下文上限**。可学习位置编码的 P 只有 $T_{\max}$ 行；输入超过这个长度就必须截断、外推或改用 RoPE。
- **tokenizer 不一致是最隐蔽的 bug**。训练用 A 词表、推理用 B 词表，模型不会报错，只会输出乱码；上线前必须校验 tokenizer 的哈希。

3.8 本章小结

- Tokenizer 把文本变成整数 ID；字符级简单，BPE 是工业标准。
- BPE 反复合并最高频的相邻对；本书语料上 24 轮合并把 24 个字符压成 13 个 token。
- Embedding 是 $V \times D$ 的查表矩阵；每个 token 得到一行 D 维向量。
- 位置编码告诉模型顺序；本书用可学习位置，现代模型常用 RoPE。
- 文本 $\to$ ID $\to [T, D]$ 矩阵，这是 Transformer 的输入。

第 4 章

注意力：Q、K、V 到底在做什么

一句话

注意力是一次“带权重的查表”：每个位置用 Query 去和所有位置的 Key 做点积，得到匹配分数；softmax 把分数变成权重；再用权重对所有位置的 Value 加权求和。因果掩码保证位置 t 只能看 t 及以前。

4.1 为什么需要上下文

如果模型只看当前 token，它无法区分“bank”是河岸还是银行，也无法知道代词指谁。模型需要把前面所有 token 的信息“读”进来。循环神经网络（RNN）逐个读，速度慢且长距离遗忘；注意力让每个位置直接和所有位置交互，一步建立全局依赖。

注意力的核心问题是：**当前位置应该看哪些位置，看多重？**这个问题被拆成三个角色：

- **Query（查询）**：我在找什么？
- **Key（键）**：我提供什么特征，供别人匹配？
- **Value（值）**：如果别人匹配上我，我交给它什么内容？

用图书馆打比方：Query 是你的搜索词，Key 是每本书的标题/标签，Value 是书的内容。点积算“搜索词和标签有多匹配”，softmax 把匹配度变成借阅权重，加权求和就是“把相关书的内容按相关度混合后交给你”。

4.2 一个 3 个 token 的手算

为了让数字可以手算，假设 $D = 2$、单头，并且 Q、K、V 的投影矩阵都是单位矩阵（真实模型里它们是学出来的）。三个 token 的向量是：

$$\mathbf{x}_1 = (1, 0), \quad \mathbf{x}_2 = (0, 1), \quad \mathbf{x}_3 = (1, 1).$$

于是 $q_i = k_i = v_i = x_i$。

4.2.1 第一步：算分数

分数矩阵 $S = QK^\top$，$S_{ij} = q_i \cdot k_j$：

$$S = \begin{pmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \\ 1 & 1 & 2 \end{pmatrix}.$$

例如 $S_{23} = q_2 \cdot k_3 = (0,1) \cdot (1,1) = 1$。

除以 $\sqrt{D} = \sqrt{2} \approx 1.414$：

$$S/\sqrt{2} = \begin{pmatrix} 0.707 & 0 & 0.707 \\ 0 & 0.707 & 0.707 \\ 0.707 & 0.707 & 1.414 \end{pmatrix}.$$

4.2.2 第二步：因果掩码

位置 1 只能看位置 1；位置 2 能看 1、2；位置 3 能看 1、2、3。把未来位置的分数设成 $-\infty$：

$$\tilde{S} = \begin{pmatrix} 0.707 & -\infty & -\infty \\ 0 & 0.707 & -\infty \\ 0.707 & 0.707 & 1.414 \end{pmatrix}.$$

4.2.3 第三步：softmax

对每一行做 softmax。第一行只有一个有效项，权重 $[1, 0, 0]$。第二行：

$$[e^0, e^{0.707}] = [1, 2.028], \quad \text{和} = 3.028, \quad [0.330, 0.670].$$

第三行：

$$[e^{0.707}, e^{0.707}, e^{1.414}] = [2.028, 2.028, 4.113], \quad \text{和} = 8.169,$$

$$[0.248, 0.248, 0.504].$$

所以注意力权重矩阵是

$$A = \begin{pmatrix} 1 & 0 & 0 \\ 0.330 & 0.670 & 0 \\ 0.248 & 0.248 & 0.504 \end{pmatrix}.$$

4.2.4 第四步：加权求和

输出 $O = AV$：

$$O_1 = 1 \cdot v_1 = (1, 0),$$

$$O_2 = 0.330(1,0) + 0.670(0,1) = (0.330, 0.670),$$

$$O_3 = 0.248(1,0) + 0.248(0,1) + 0.504(1,1) = (0.752, 0.752).$$

第一个 token 完全看不到后面，所以它的输出就是自己的 value；第三个 token 综合了三个位置。

手算

把上面的数字代回公式：

$$\text{Attention}(Q,K,V)=\text{softmax}\left(\frac{QK^\top}{\sqrt{D}}+M\right)V,$$

其中 $M_{ij}=0$（$j\le i$）或 $-\infty$（$j>i$）。三个矩阵乘、一次 softmax、一次加权和，就是注意力的全部。

4.3 为什么要除以 $\sqrt{D}$

假设 q_i,k_i 的每个分量独立、均值 0、方差 1。点积

$$q\cdot k=\sum_{i=1}^{D}q_ik_i$$

的均值是 0，方差是 D（D 个独立项相加）。$D=4096$ 时，点积的标准差是 64；如果不缩放，softmax 的输入会很大，指数后几乎变成 one-hot，梯度消失，训练不稳定。

除以 $\sqrt{D}$ 后，方差变回 1，softmax 的输入落在合理区间。这就是公式里 $\sqrt{D}$ 的来历，不是随便选的常数。

手算

$D=2$ 时，点积方差约 2，除以 $\sqrt{2}$ 后方差约 1。$D=4096$ 时，点积标准差约 64，除以 64 后方差约 1。量级对齐，softmax 才不会饱和。

4.4 多头注意力

单头只能学一种“相关性”。多头把 D 维切成 H 份，每份 $d_h=D/H$，每个头独立做一次注意力，最后把结果拼接，再过一个输出投影：

$$\text{MHA}(X)=\text{Concat}(\text{head}_1,\dots,\text{head}_H)W_O. \tag{4.1}$$

不同的头可以关注不同的模式：有的头看相邻词，有的头看句法依赖，有的头看标点。头数增加，计算量不变（总维度还是 D），但表达更丰富。

结构	头数	每头维度	本书玩具
MHA	H	D/H	1 头，24 维
GQA	H 个 Q，G 组 KV	D/H	推理常用，省 KV Cache
MQA	H 个 Q，1 组 KV	D/H	最省，效果略降

GQA/MQA 只改变 K/V 的头数，不改变 Q 的头数；它直接决定 KV Cache 的大小（第 6 章）。

4.5 Transformer Block：注意力之外还有什么

一个 block 由两部分组成：

$$\mathbf{x}' = \mathbf{x} + \text{Attention}(\text{LN}(\mathbf{x})), \tag{4.2}$$

$$\mathbf{x}'' = \mathbf{x}' + \text{MLP}(\text{LN}(\mathbf{x}')). \tag{4.3}$$

注意残差连接：每个子层的输出加回输入。残差让梯度有一条“高速公路”直接回传，深层网络才训得动。

4.5.1 LayerNorm 手算

对一行向量 $\mathbf{x} = (x_1, \ldots, x_D)$：

$$\mu = \frac{1}{D}\sum_i x_i, \qquad \sigma^2 = \frac{1}{D}\sum_i (x_i - \mu)^2, \tag{4.4}$$

$$\text{LN}(\mathbf{x})_i = \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}}\gamma_i + \beta_i. \tag{4.5}$$

γ, β 是可学习参数，初始 $\gamma = 1, \beta = 0$。

手算

$\mathbf{x} = (1, 2, 3)$，$\epsilon = 0$：

$$\mu = 2, \quad \sigma^2 = \frac{(1-2)^2 + 0 + (3-2)^2}{3} = \frac{2}{3}, \quad \sigma = 0.816.$$

$$\text{LN}(\mathbf{x}) = \left(\frac{-1}{0.816}, 0, \frac{1}{0.816}\right) = (-1.225, 0, 1.225).$$

均值变 0，方差变 1。LayerNorm 让每层的输入尺度稳定，学习率可以设得更大。

RMSNorm 是简化版：不减均值，只除以均方根：

$$\text{RMSNorm}(\mathbf{x})_i = \frac{x_i}{\sqrt{\frac{1}{D}\sum_j x_j^2 + \epsilon}}\gamma_i. \tag{4.6}$$

它少一次减均值，速度略快，LLaMA 等模型常用。

4.5.2 MLP 手算

MLP 是两层线性变换加非线性：

$$\text{MLP}(\mathbf{x}) = W_2\,\sigma(W_1\mathbf{x} + b_1) + b_2, \tag{4.7}$$

其中 σ 是 ReLU：$\max(0, x)$。W_1 把 D 维升到 D_{ff}（通常是 $4D$），W_2 再降回 D。

手算

$D=2, D_{ff}=3$，$W_1=\begin{pmatrix}1&0\\0&1\\1&1\end{pmatrix}$，$b_1=0$，$\mathbf{x}=(1,-2)$：

$$W_1\mathbf{x}=(1,-2,-1),\quad \text{ReLU}=(1,0,0).$$

如果 $W_2=(1,1,1)$，输出是 1。负数和被截断，只保留正信号。

现代模型常用 SwiGLU：$\text{Swish}(W_1x)\odot(W_3x)$，再多一个门控矩阵，效果更好但参数更多。本书用 ReLU，便于手算。

4.6 本书玩具模型的完整前向

把上面的零件装起来，形状变化如表 4.1。

步骤	公式	形状
输入	X	$[T,24]$
LN1	$H=\text{LN}(X)$	$[T,24]$
Q/K/V	HW_q, HW_k, HW_v	$[T,24]$ 各一个
分数	$S=QK^\top/\sqrt{24}$	$[T,T]$
掩码	$S_{ij}=-\infty$ if $j>i$	$[T,T]$
权重	$A=\text{softmax}(S)$	$[T,T]$
注意力	$Y=AV$	$[T,24]$
输出投影	YW_o	$[T,24]$
残差 1	$X_1=X+YW_o$	$[T,24]$
LN2	$H_2=\text{LN}(X_1)$	$[T,24]$
MLP	$H_3=\text{ReLU}(H_2W_1)W_2$	$[T,24]$
残差 2	$X_2=X_1+H_3$	$[T,24]$
最终 LN	$H_f=\text{LN}(X_2)$	$[T,24]$
logits	H_fW_{lm}	$[T,16]$

表 4.1：一个 block 的形状变化。$T=32$，$D=24$，$D_{ff}=48$，$V=16$。

参数可以逐项相加（表 4.2），总共 6472 个。这个数字会在第 10、11 章反复出现。

4.7 代码走查

4.7.1 训练/预填：一次算完整个序列

Listing 4.1：因果注意力的前向（py/model.py 节选）

```
h = layernorm(x, params["ln1_g"], params["ln1_b"])
q = add(matmul(h, params["wq"]), params["bq"])
k = add(matmul(h, params["wk"]), params["bk"])
```

参数	形状	数量
token embedding	16×24	384
position embedding	32×24	768
LN1 γ, β	$24 + 24$	48
W_q, b_q	$24 \times 24 + 24$	600
W_k, b_k	$24 \times 24 + 24$	600
W_v, b_v	$24 \times 24 + 24$	600
W_o, b_o	$24 \times 24 + 24$	600
LN2 γ, β	$24 + 24$	48
W_1, b_1	$24 \times 48 + 48$	1200
W_2, b_2	$48 \times 24 + 24$	1176
LNf γ, β	$24 + 24$	48
lm_head, bias	$24 \times 16 + 16$	400
合计		6472

表 4.2: 玩具模型的参数账本。

```
v = add(matmul(h, params["wv"]), params["bv"])
scores = scale(matmul(q, transpose(k)), 1.0 / math.sqrt(cfg.n_embd))
scores = masked_fill(scores, causal_mask(t), -1e9)
attn = softmax_rows(scores)
y = matmul(attn, v)
y = add(matmul(y, params["wo"]), params["bo"])
x = add(x, y)
```

这段代码和手算逐行对应：`matmul(q, transpose(k))` 算 $QK^\top$，`scale` 除以 $\sqrt{D}$，`masked_fill` 把未来位置设成 -10^9，`softmax_rows` 按行归一，`matmul(attn, v)` 加权求和。

4.7.2 Decode：只有一个新 token

Decode 时，Q 只有一行，K/V 是缓存里的所有历史行：

Listing 4.2: 带 KV Cache 的单步注意力（`py/infer.py` 节选）

```
q, k, v = qkv(params, x, cfg)
cache.k.append(k[0][:])
cache.v.append(v[0][:])
scores = [sum(q[0][z] * kk[z] for z in range(d)) * scale
          for kk in cache.k]
probs = softmax(scores)
out = [[sum(probs[j] * cache.v[j][z] for j in range(len(cache.v)))
        for z in range(d)]]
```

没有因果掩码：新 token 在最后，它能看到所有历史，历史看不到它。缓存让 K, V 不必重算，这是第 6 章的主题。

4.7.3 训练时为什么要掩码

训练时整条序列一次算完，位置 t 的分数行包含 $t+1,\ldots,T$ 的未来 token。如果不掩码，模型会“偷看答案”，训练 loss 很低，推理却完全不会。因果掩码是训练和推理一致性的保证。

4.8 常见误解

常见坑

- **Q、K、V 不是三个不同的输入**。它们都是同一个 X 经过三个不同的线性投影得到的；投影矩阵是学出来的。
- **注意力不是“可解释的权重”**。 softmax 权重高只说明当前层的匹配度高，不等于人类语义上的“重要”。
- **多头不是把模型变大**。总维度不变，只是把维度分给多个头；计算量基本相同。
- **除以 $\sqrt{D}$ 不是归一化**。它只调整方差，不改变向量方向；LayerNorm 才是归一化。
- **因果掩码只在训练/预填时需要**。 Decode 单步不需要掩码，因为新 token 本来就在最后。

4.9 本章小结

- 注意力 = 用 Q 和 K 的点积算匹配，softmax 变权重，再对 V 加权求和。
- 除以 $\sqrt{D}$ 是为了让点积方差回到 1，避免 softmax 饱和。
- 因果掩码保证位置 t 只看到 t 及以前；训练和推理必须一致。
- 多头把维度切分，分别学习不同的相关性；GQA/MQA 通过减少 KV 头数省显存。
- Block = 注意力子层 + MLP 子层，每个子层都有残差和 LayerNorm。
- 玩具模型 6472 个参数，形状和公式都可以手算。

第 5 章

从 Logits 到字符：采样

一句话

模型每步输出的不是字，而是词表上的一个概率分布。采样就是从这个分布里挑一个 token。temperature 改变分布的陡峭程度，top-k/top-p 砍掉长尾。采样只影响“挑哪个”，不影响前面的 Transformer 计算和 KV Cache。

5.1 Logits 到概率

模型最后一层输出 V 个 logits $z_1, \dots, z_V$。softmax 把它们变成概率：

$$p_i = \frac{e^{z_i}}{\sum_j e^{z_j}}. \tag{5.1}$$

本书 $V = 16$，所以每步得到一个 16 维分布。取概率最大的 token 叫贪心（greedy）；按概率随机抽叫采样（sampling）。

手算

某步 logits 为 $(2.0, 1.0, 0.5, 0.0)$。减去最大值 2 后：

$$e^0 = 1, \quad e^{-1} = 0.368, \quad e^{-1.5} = 0.223, \quad e^{-2} = 0.135,$$

和为 1.726，概率为

$$(0.579, 0.213, 0.129, 0.078).$$

贪心选第 1 个；采样有 21.3% 的概率选第 2 个。

5.2 四种常见采样

5.2.1 贪心（temperature = 0）

每步取 $\arg\max_i p_i$。确定性最强，适合代码、结构化抽取、需要可复现的场景。缺点是容易重复、死循环，创造力弱。

5.2.2 Temperature

把 logits 除以温度 T，再 softmax：

$$p_i(T) = \frac{e^{z_i/T}}{\sum_j e^{z_j/T}}. \tag{5.2}$$

$T < 1$ 让分布更尖（更确定），$T > 1$ 让分布更平（更随机），$T \to 0$ 退化为贪心。

手算

对上面的 logits $(2, 1, 0.5, 0)$：

$$T = 0.5: \ (0.760, 0.168, 0.062, 0.010),$$

$$T = 1.0: \ (0.579, 0.213, 0.129, 0.078),$$

$$T = 1.5: \ (0.474, 0.249, 0.170, 0.107).$$

温度越低，最大概率越接近 1；温度越高，尾部越厚。

5.2.3 Top-k

只保留概率最高的 k 个 token，重新归一化，再采样。它把明显不合理的尾部直接砍掉，避免“抽风”。$k = 1$ 等价于贪心；$k = V$ 等价于纯采样。

5.2.4 Top-p（nucleus）

按概率从大到小排序，取累计概率达到 p 的最小集合，重新归一化。它比 top-k 更自适应：分布尖时集合小，分布平时集合大。

手算

概率 $(0.5, 0.25, 0.15, 0.10)$，$p = 0.8$：

$$0.5 + 0.25 + 0.15 = 0.90 \geq 0.8,$$

所以保留前 3 个，重新归一化为 $(0.556, 0.278, 0.167)$。第 4 个被砍掉。

5.3 随机数与可复现性

采样需要随机数。Python 的 `random.Random(seed)` 用 Mersenne Twister；Go 的 `math/rand` 算法不同。同样的 seed、同样的概率，两个语言抽出的序列可能不同。本书的 Go 实现用 splitmix64，Python demo 用 MT；因此：

- 贪心（$T = 0$）在两边完全一致；
- 采样（$T > 0$）只保证分布一致，不保证逐 token 相同。

如果要在两边逐 token 对齐，需要实现同一个 RNG，或者只比较概率/贪心结果。第 7 章的对拍用的是贪心。

5.4 本书 demo 的真实输出

提示词 "the cat "，贪心解码，Python 输出：

```
prefill: 8 tokens in 1.039 ms; first token 'r'; cache 8 tokens, 3072 bytes
  top-5: 'r':0.3276, 's':0.3245, '.':0.2890, 'o':0.0393, 'l':0.0105
...
  10   'l'   0.5228     19   0.163  'l':0.523, 'm':0.299, 'd':0.115
...
output: 'the cat ran to the log .'
```

第一步的 top-1 是 r（0.3276），top-2 是 s（0.3245），只差 0.003。贪心选 r，得到 ran；如果采样，有 32.45% 的概率选 s，得到 sat。两种都是合法续写。这解释了为什么同一个模型、同一个 prompt，不同温度会得到不同答案。

温度 $T = 0.7$ 时，同一位置的 top-5 变成：

$$\text{r} : 0.3471, \quad \text{s} : 0.3425, \quad . : 0.2903, \quad \text{o} : 0.0168, \quad \text{l} : 0.0026.$$

分布变平了一点，r 和 s 的差距更小，随机性更强。

5.5 代码

Listing 5.1: 采样（py/infer.py 节选）

```python
def sample(logits, temperature, top_k, top_p, rng):
    if temperature <= 0:
        probs = softmax(logits)
        return max(range(len(probs)), key=lambda i: probs[i]), top5
    scaled = [z / temperature for z in logits]
    order = sorted(range(len(scaled)), key=lambda i: -scaled[i])
    if top_k > 0:
        order = order[:top_k]
    probs = softmax([scaled[i] for i in order])
    if top_p < 1.0:
        keep, acc = [], 0.0
        for i, p in zip(order, probs):
            keep.append(i); acc += p
            if acc >= top_p: break
        order = keep
        probs = softmax([scaled[i] for i in order])
    r = rng.random(); acc = 0.0
    for i, p in zip(order, probs):
        acc += p
        if r <= acc:
            return i, top5
```

注意最后一步：不是“取最大”，而是“按累积概率抽”。这样概率高的 token 更可能被选中，但低概率 token 仍有机会。

5.6 工程视角

工程视角

- **确定性是一种可缓存、可省钱的资源。** $T = 0$ 时相同前缀必然产生相同输出，前缀缓存、响应缓存都能无损命中；$T > 0$ 时缓存价值下降。
- **采样参数不影响 KV Cache 与批处理结构。**不同 temperature、top-p 的请求可以混在一个 batch 里共享权重读取；采样只发生在最后一步。
- **max_tokens 是资源参数。**它决定请求占用 KV Cache 槽位的时长；调大它等于让长尾请求占住稀缺显存。
- **流式输出是指标的一部分。** TPOT 的定义依赖逐 token 推送；把响应攒到最后一次性返回，用户感知的是端到端延迟，TTFT 指标形同虚设。
- **结构化输出需要约束。**如果要 JSON/代码，贪心或低温度更稳；也可以在每个 token 上做 grammar mask，把非法 token 的概率设为 0。

5.7 本章小结

- 模型输出 logits，softmax 变概率，采样选 token。
- 贪心确定；temperature 调陡峭；top-k/top-p 截尾。
- 采样不改变 Transformer 计算和 KV Cache，只改变最后一步。
- 贪心可跨语言逐 token 对拍；采样只保证分布一致。
- 确定性、max_tokens、流式输出都是工程参数，不只是“算法参数”。

第 6 章

KV Cache：把重复计算变成一次读取

一句话

Decode 第 t 步只需要新 token 的 Q，但它要和前 $t-1$ 个 token 的 K、V 做注意力。K、V 不随新 token 改变，所以可以缓存。缓存把“每步重算整段历史”变成“每步只算一个新 token + 读历史”，代价是显存。

6.1 没有缓存时，Decode 在重复算什么

自回归生成第 t 个 token 时，模型需要前 t 个 token 的表示。最笨的办法是：把前 t 个 token 重新跑一遍完整前向。第 t 步的计算量包括：

- 所有 token 的 Q/K/V 投影：$O(tD^2)$；
- 所有 token 的 MLP：$O(tD^2)$；
- 注意力分数 $t \times t$：$O(t^2D)$。

生成 n 个 token，把 $t = 1, \ldots, n$ 加起来：

$$\sum_{t=1}^{n} O(tD^2 + t^2D) = O(n^2D^2 + n^3D). \tag{6.1}$$

其中 n^2D^2 来自“每步都重算历史 token 的投影和 MLP”，n^3D 来自注意力的平方增长。

有了 KV Cache，第 t 步只算新 token：

- 新 token 的 Q/K/V 投影：$O(D^2)$；
- 新 token 的 MLP：$O(D^2)$；
- 新 Q 与缓存里 t 个 K 做点积：$O(tD)$。

总和：

$$\sum_{t=1}^{n} O(D^2 + tD) = O(nD^2 + n^2D). \tag{6.2}$$

最大的 n^2D^2 项消失了；注意力的 n^2D 项还在（每个新 token 仍要看所有历史），但它的系数小得多。对 7B 模型（$D = 4096$），$D^2 \approx 1.7 \times 10^7$，$D \approx 4 \times 10^3$，差四个数量级。

6.2 缓存里到底存什么

对每一层、每个 token，缓存两个向量：

- $K_t = W_k \mathbf{h}_t$：这个 token 的“键”，供后面的 Query 匹配；
- $V_t = W_v \mathbf{h}_t$：这个 token 的“值”，被匹配后提供内容。

不缓存 Q，因为 Q 只属于当前 token，下一步就换了。也不缓存注意力权重，因为每一步的 Query 不同，权重不同。

6.2.1 每个 token 的缓存大小

设层数 L、KV 头数 H_{kv}、每头维度 d_h、每个数 b 字节。每个 token 在每层存 K 和 V 两份，每份 $H_{kv} d_h$ 个数：

$$\text{bytes/token} = 2 \times L \times H_{kv} \times d_h \times b. \tag{6.3}$$

本书玩具模型：$L = 1, H_{kv} = 1, d_h = 24$，float64 的 $b = 8$：

$$2 \times 1 \times 1 \times 24 \times 8 = 384 \text{ B/token}.$$

生成 23 个 token，缓存 $23 \times 384 = 8832$ B，和日志里的 8832 完全一致。

7B 模型：$L = 32, H_{kv} = 8, d_h = 128$，bf16 的 $b = 2$：

$$2 \times 32 \times 8 \times 128 \times 2 = 131072 \text{ B} = 128 \text{ KiB/token}.$$

4096 token 的单条序列：

$$128 \text{ KiB} \times 4096 = 512 \text{ MiB}.$$

结构	KV 头数	每 token	4096 上下文单序列
MHA	32	512 KiB	2 GiB
GQA（本书 7B 算例）	8	128 KiB	512 MiB
MQA	1	16 KiB	64 MiB

表 6.1: KV Cache 随 KV 头数线性缩小。GQA 把 KV 头从 32 降到 8，缓存直接除以 4。

6.2.2 为什么 GQA 能省这么多

MHA 里每个 Query 头都有自己的一组 K/V；GQA 让多个 Query 头共享一组 K/V。比如 32 个 Query 头、8 组 KV，每 4 个 Query 头共用一组 KV。注意力的计算量基本不变，但缓存和读取量除以 4。这是模型结构层面为推理省显存的标准做法。

6.3 Prefill 与 Decode 的两种账本

6.3.1 Prefill：一次算完，计算密集

Prefill 把 T 个 prompt token 拼成矩阵，一次前向：

- 计算量约 $2NT$（N 为参数量）；
- 访存量约“权重读一遍”$2N$ 字节（bf16）；
- 算术强度约 T FLOP/B。

$T = 1000$ 时，$I \approx 1000$，远高于 H100 的拐点 333，是计算密集。

6.3.2 Decode：一次一个，访存密集

Decode 每步只来 1 个 token：

- 计算量约 $2N$；
- 访存量约“权重读一遍”$2N$ 字节 + KV 读取；
- 算术强度约 1 FLOP/B。

$I \approx 1$，远低于 333，是访存密集。算力再强，也要等权重从 HBM 读出来。

手算

7B 模型 bf16 权重 14 GB，HBM 带宽 3 TB/s：

$$t_{\text{weight}} = \frac{14\ \text{GB}}{3\ \text{TB/s}} \approx 4.7\ \text{ms/token}.$$

单请求 Decode 的理论上限约 $1/0.0047 \approx 213$ token/s，与 GPU 的算力无关，只取决于带宽。上下文 4096 时还要读 512 MiB KV：

$$t_{\text{kv}} = \frac{0.5\ \text{GB}}{3\ \text{TB/s}} \approx 0.17\ \text{ms},$$

相对权重读取还小；上下文到 128k 时，KV 读取会超过权重，成为第一瓶颈。

6.4 把缓存写出来

缓存就是一个按 token 增长的 K、V 列表。Prefill 时把整段 prompt 的 K/V 追加进去；Decode 时每步追加一行，然后用新 Q 和全部缓存做注意力。

Listing 6.1: KV Cache 的数据结构与单步注意力（py/infer.py 节选）

```
@dataclass
class KVCache:
    k: list[list[float]] = field(default_factory=list)
    v: list[list[float]] = field(default_factory=list)
    bytes_per_value: int = 8

def attn_decode(params, x, cache, cfg):
    q, k, v = qkv(params, x, cfg)
    d = cfg.n_embd
    cache.k.append(k[0][:])
    cache.v.append(v[0][:])
    scores = [sum(q[0][z] * kk[z] for z in range(d)) * scale
```

```
            for kk in cache.k]
    probs = softmax(scores)
    out = [[sum(probs[j] * cache.v[j][z] for j in range(len(cache.v)))
            for z in range(d)]]
    return out
```

Prefill 的注意力要一次处理 T 个 Query，所以分数是 $T \times T$，需要因果掩码；Decode 只有 1 个 Query，分数是 $1 \times (t+1)$，不需要掩码，因为新 token 本来就在最后。

6.5 对拍：缓存必须和全量重算完全一致

缓存是一个优化，不能改变结果。验证方法：每生成一个 token，用缓存算一次 logits；再把整段序列从头跑一遍，算同样的 logits；比较最大绝对差。本书的 Python 和 Go demo 都做了这件事：

```
verification: compare cached decode with full recomputation
  pos  8: max|cached-full| = 0.000e+00
  ...
  max diff over checked positions = 0.000e+00
timing (24 decode steps): cached=3.95 ms, full-recompute=73.69 ms,
                          speedup=18.68x
```

差值恰好为 0，因为两条路径的浮点运算顺序完全相同。实际系统里可能有 10^{-6} 量级的差异，来自 kernel 的归约顺序；只要在容差内，就说明缓存逻辑正确。

实现	prefill 8 token	decode 每 token	24 步加速比
Python（float64，纯解释器）	1.08 ms	0.156 ms	18.7×
Go（float64，编译执行）	0.063 ms	0.007 ms	19.6×

表 6.2: 本书 demo 的实测。Python 的绝对时间主要花在解释器和列表分配上，Go 更接近“纯计算”。

6.6 显存墙：一张卡能放多少条序列

7B 模型推理，bf16 权重 14 GB。80 GB 卡扣掉权重、框架和碎片，假设剩 56 GB 给 KV：

$$\frac{56\ \text{GB}}{512\ \text{MiB}} \approx 112\ \text{条 4096 token 序列.}$$

如果平均上下文只有 512 token，每条只占 64 MiB，能放约 896 条。真实系统不会按最大长度预留，而是按需分配（第 8 章的 PagedAttention）。

批量放大：batch 32、每条 4k 上下文，KV 共

$$32 \times 512\ \text{MiB} = 16\ \text{GiB},$$

已经超过权重本身。Decode 每步要读 14 GB 权重 + 16 GB KV，共 30 GB；按 3 TB/s 约 10 ms。32 个 token / 10 ms，吞吐约 3200 token/s。这就是“攒 batch”的收益：权重只读一遍，摊到 32 个请求上。

6.7 算术强度：为什么 batch 能救 Decode

单请求 Decode 的算术强度约 1 FLOP/B，远低于拐点；batch B 时，权重读一遍服务 B 个 token，算术强度近似乘 B：

$$I_B \approx \frac{2NB}{2N + 2LTH_{kv}d_h b} \approx B \quad (\text{当 KV 远小于权重时}). \tag{6.4}$$

$B = 32$ 时 $I \approx 32$，仍低于 333，但已经大幅改善；B 大到 KV 读取占主导时，再增加 batch 收益递减。这就是推理引擎要在“吞吐”和“TPOT”之间调 batch 的原因。

6.8 工程视角

工程视角

- **KV Cache 是一等资源**。它的总量是硬约束，分配效率决定并发上限；PagedAttention、前缀缓存、KV 量化都围绕它。
- **缓存不是免费的**。每条序列、每个 token 都占显存；长上下文、多轮对话会把缓存推到权重之上。
- **缓存可以量化**。 bf16 到 int8 直接减半，代价是轻微精度损失；这是线上常用手段。
- **缓存可以共享**。相同 system prompt 的多个请求可以共享前缀的 KV 块，参考计数管理；第 8 章展开。
- **缓存会失效**。模型权重、tokenizer、位置编码、RoPE 参数任一改变，旧缓存都不能用；版本要绑定。

6.9 练习

1. 把本书模型的 `bytes_per_value` 改成 2（模拟 bf16），23 个 token 的缓存是多少字节？
2. 7B 模型、32 层、8 个 KV 头、$d_h = 128$、int8 缓存，每 token 多少字节？4k 上下文单序列多少？
3. 如果 batch 从 1 增到 64，Decode 每步的权重读取量变不变？KV 读取量变多少？算术强度大致变多少？
4. 把 `attn_decode` 里的 `cache.k.append` 注释掉，运行会发生什么？为什么？

第 7 章

完整跑通：Python 与 Go 单机 demo

一句话

这一章不引入新概念，只把前面的零件装起来，给你一份可以照着敲、照着改、照着对拍的运行手册。跑完之后，你应该能指着任意一行输出说清楚它是哪一步算出来的。

7.1 目录与两条命令

代码在本书配套的 `llmbook/` 下，结构见附录 C。最短路径只有两条：

```
# Python：训练 600 步，再贪心推理
python3 py/train.py --steps 600 --batch 4 --out weights/tiny_py.json
python3 py/infer.py --temperature 0

# Go：训练 1200 步，再用 Python 的权重推理
cd go
go run ./cmd/train -steps 1200 -out ../weights/tiny_go.json
go run ./cmd/infer -weights ../weights/tiny_py.json -temp 0
```

文件	作用
`py/autograd.py`	反向模式自动微分：Tensor、算子、backward
`py/model.py`	微型 Transformer 的前向、参数初始化、存取权重
`py/train.py`	数据采样、损失、AdamW、warmup/cosine、训练循环
`py/infer.py`	Prefill、KV Cache、Decode、采样、对拍
`py/neuron.py`	2 参数神经元的手算与梯度下降
`go/tiny/model.go`	Go 版前向、KV Cache、采样、JSON 加载
`go/tiny/train.go`	手写反向传播、AdamW、保存权重
`go/cmd/infer、train、neuron`	三个命令行入口

表 7.1：代码阅读地图。建议顺序：neuron → model → infer → train。

两条命令都不需要安装依赖。Python 训练约 100 秒，Go 训练约 2 秒；推理都在毫秒级。

7.2 Python 输出逐行读

下面是 `python3 py/infer.py --temperature 0` 的真实输出（省略了部分重复行）：

```
model: 1 layer, 1 head, d_model=24, ff=48, vocab=16, block=32
params: 6472
prompt: 'the cat ' -> 8 tokens [14, 7, 5, 0, 3, 2, 14, 0]

prefill: 8 tokens in 1.080 ms; first token 'r'; cache 8 tokens, 3072 bytes (float64)
  top-5: 'r':0.3276, 's':0.3245, '.':0.2890, 'o':0.0393, 'l':0.0105

step token  p(top1)  cache      ms  top-3
   0   'a'   0.9979      9   0.156  'a':0.998, ' ':0.001, 'o':0.000
   1   'n'   0.9891     10   0.158  'n':0.989, 't':0.006, 'w':0.003
   2   ' '   0.9998     11   0.158  ' ':1.000, 'g':0.000, 'a':0.000
...
  10   'l'   0.5228     19   0.163  'l':0.523, 'm':0.299, 'd':0.115
...
  14   '.'   0.9771     23   0.161  '.':0.977, 'r':0.006, 's':0.005

output: 'the cat ran to the log .'
decode: 15 steps, mean 0.156 ms/token, total 2.339 ms
KV cache final: 23 tokens, 8832 bytes (8.6 KiB, float64)

verification: compare cached decode with full recomputation
  pos  8: max|cached-full| = 0.000e+00
...
  max diff over checked positions = 0.000e+00

timing (24 decode steps): cached=3.95 ms, full-recompute=73.69 ms, speedup=18.68x
```

逐行解释：

1. `model...` 是配置：1 层、1 头、24 维、FFN 48、词表 16、上下文 32。
2. `params: 6472` 是参数总数，和表 4.2 的逐项相加一致。
3. `prompt...` 是 tokenizer 的结果：8 个整数，对应 `t h e c a t ` 。
4. `prefill...` 是一次并行前向：8 个 token 一起算，耗时 1.039 ms，得到第一个 token `r`，同时建立 8 个 token 的 KV Cache。
5. `top-5` 是第一个 token 的概率分布：`r` 0.3276，`s` 0.3245，`.` 0.2890。三者的概率很接近，所以贪心和采样的结果可能不同。
6. 表格每一行是一次 Decode：新 token、它的 top-1 概率、当前缓存长度、本步耗时、top-3。第 0 步生成 `a`，第 1 步 `n`，拼起来是 `ran`。
7. 第 10 步的 top-1 只有 0.5228，`m` 有 0.299，说明模型在 `l` 和 `m` 之间犹豫；贪心选了 `l`，最后得到 `log`。
8. `output` 是最终文本；`decode` 是 15 步的统计；`KV cache final` 是 23 个 token 的缓存，每个 token 384 B，共 8832 B。
9. `verification` 是缓存与全量重算的对拍，差值全为 0。
10. `timing` 是 24 步缓存 vs 24 步全量重算：3.89 ms vs 73.82 ms，约 19 倍。

7.3 Go 输出与对拍

Go 的同一段推理（用 Python 训练出的权重）：

```
model: 1 layer, 1 head, d_model=24, ff=48, vocab=16, block=32
params: 6472
prompt: "the cat " -> 8 tokens [14 7 5 0 3 2 14 0]

prefill: 8 tokens in 0.063 ms; first token "r"; cache 8 tokens, 3072 bytes (float64)
  top-5: "r":0.3276, "s":0.3245, ".":0.2890, "o":0.0393, "l":0.0105
...
output: "the cat ran to the log ."
decode: 15 steps, mean 0.007 ms/token, total 0.109 ms
...
timing (24 decode steps): cached=0.20 ms, full-recompute=3.98 ms, speedup=19.62x
```

两边的前 5 个概率、生成的 15 个 token、缓存的 8832 B、对拍差值全部一致。反过来也成立：用 Go 训练出的 `tiny_go.json`，Python 推理同样能加载，输出仍是 `the cat ran to the log .`，概率也逐位一致。这不是巧合：Go 代码逐行复刻了 Python 的运算顺序（同样的矩阵乘累加顺序、同样的 softmax、同样的 float64），所以连浮点结果都相同。

指标	Python	Go	倍率
prefill 8 token	1.08 ms	0.063 ms	17×
decode 每 token	0.156 ms	0.007 ms	22×
24 步缓存	3.95 ms	0.20 ms	20×
24 步全量重算	73.69 ms	3.98 ms	19×
缓存加速比	18.7×	19.6×	–

表 7.2: 同一算法、同一权重、同一台 M4 Pro 的实测。绝对时间每次运行会波动，倍率关系稳定。

本书的实测环境：Apple M4 Pro（arm64）、macOS、Python 3.13.7、Go 1.26.0；两者都是纯 float64、无 SIMD/GPU 优化。换一台机器，绝对时间会变，倍率关系不变。

7.3.1 为什么 Go 快这么多

不是 Go 的矩阵乘有多优化，而是 Python 的解释器开销太大：

- 每次 `for` 循环、每个 `list` 下标、每次函数调用都要经过解释器；
- Python 的 `list[list[float]]` 是“指针的指针”，缓存局部性差；
- Go 的 slice 是连续内存，循环是编译后的机器码，CPU 缓存命中率高。

两者都是 $O(\cdot)$ 相同的算法，常数差 20–40 倍。生产推理引擎用 C++/CUDA 写 kernel，再用 Python 做调度，就是为了拿到这个常数。

7.3.2 为什么缓存加速比不是 100 倍

24 步全量重算的每一步长度不同 (8,9,...,31)，平均约 20 个 token；缓存每一步只算 1 个 token 的投影 + 读历史。理论加速比约 20 倍，和实测 19–28 倍吻合。如果序列更长、模型更大，加速比会更明显，因为全量重算的 $O(tD^2)$ 项占主导。

7.4 你可以马上做的四个实验

1. **改温度**。`python3 py/infer.py --temperature 0.7 --top-k 6`。观察第一 token 的概率变平，输出可能从 `ran` 变成 `sat`。
2. **改提示词**。`--prompt "the dog "`，输出应变成 `the dog sat on the log .` 一类。
3. **改层数**。在 `py/model.py` 的 `Config` 里把 `n_layer` 改成 2；参数量从 6472 变成约 12000，训练变慢，但代码路径不变。
4. **破坏缓存**。在 `attn_decode` 里注释掉 `cache.k.append(...)`，再跑推理。缓存不再增长，输出会立刻变乱，对拍差值不再是 0；这是理解缓存作用最快的方法。

7.5 故障排查

现象	可能原因/处理
`ModuleNotFoundError: autograd`	在 `llmbook/` 根目录运行，或检查 `sys.path`
`weights/tiny_py.json` 不存在	先运行 `py/train.py`
生成乱码	权重与词表不匹配；确认用的是同一次训练的 JSON
`max|cached-full|` 不为 0	缓存追加位置/顺序错了；检查 `pos` 与 `cache.seq_len`
Go 编译很慢/权限错误	设置 `GOCACHE` 到可写目录；Go 1.21+
Python 训练很慢	正常；纯 Python 600 步约 100 秒

7.6 从 demo 到生产：还差什么

这个 demo 已经包含生产推理的数学骨架，但少了工程外壳：

- **算子**：demo 用三重循环；生产用 cuBLAS/FlashAttention/Triton kernel。
- **批处理**：demo 一次一条序列；生产用连续批处理，把几十上百条请求拼成一个 batch。
- **显存管理**：demo 用 Python list；生产用 PagedAttention 的块池。
- **量化**：demo 用 float64；生产用 bf16/fp8/int4 kernel。
- **调度**：demo 没有队列、抢占、SLO；生产有 PD 分离、限流、路由。
- **可观测性**：demo 打印日志；生产有 TTFT/TPOT、KV 命中率、GPU 利用率。

第 8 章会把这些外壳逐个拆开，并始终用 demo 的实测数字做参照。

7.7 本章小结

- 两条命令就能跑通训练 + 推理；Python 适合读，Go 适合看性能。
- Python 和 Go 的贪心输出、概率、缓存字节、对拍差值完全一致。
- Python 慢约 20–40 倍是解释器/内存布局的常数差异，不是算法差异。
- 缓存加速比约 19–28 倍，和“全量重算 vs 单 token”的理论比一致。
- demo 是显微镜；第 8 章是望远镜。

第 8 章

推理工程：demo 之外的 90%

一句话

单机 demo 回答“一个请求怎么算”；推理工程回答“一万个请求怎么排队、怎么共享显存、怎么在 SLO 内把 GPU 喂饱”。核心矛盾只有一个：Prefill 吃算力，Decode 吃带宽，而它们共享同一张卡的显存和算力。

8.1 批处理：从静态 barrier 到连续调度

8.1.1 静态批处理

攒够 B 个请求，一起跑，等最长的请求结束才放下一批。问题是短请求结束后，它的 slot 空转，GPU 被最长的请求拖住。

本书 `py/schedule.py` 模拟了 40 个请求（1 个 100 token，39 个 10 token）、4 个 slot：

```
static:     batches=10, steps=190, slot-steps=760,
            useful=490, utilization=64.5%
continuous: steps=130, slot-steps=520, useful=490,
            utilization=94.2%
```

静态批处理浪费了 35.5% 的 slot；连续批处理只浪费 5.8%（最后长请求独自跑完的尾巴）。

8.1.2 连续批处理

把调度粒度从“请求级”细化到“iteration 级”：每跑完一步 Decode，检查谁结束了、谁在排队、显存够不够；结束的立刻释放 KV，排队的立刻插入。batch 的组成是流动的，GPU 始终有活干。



连续批处理带来两个后果：

- **TTFT 与 TPOT 解耦**。新请求插入会稍微拖慢在跑请求的 TPOT；调度器要在首 token 延迟和稳态吞吐之间调参。
- **抢占成为标配**。显存不够时，低优先级请求的 KV 被丢弃（重算）或换出到 CPU（swap），等显存宽裕再恢复。这和操作系统的换页是同一套思路。

8.1.3 Chunked Prefill

一个长 prompt 的 Prefill 会独占 GPU 几百毫秒，期间所有 Decode 请求的 TPOT 抖动。Chunked Prefill 把长 prompt 切成小块，和 Decode 请求混在一个 batch 里跑：每步带一点 Prefill、一点 Decode，TTFT 稍长，但 TPOT 稳定。

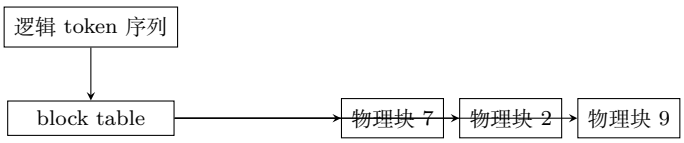
8.2 PagedAttention：给 KV Cache 做虚拟内存

8.2.1 预留式管理的浪费

传统实现按最大长度给每个请求预留连续显存。7B 模型、4k 上下文，每条预留 512 MiB；实际只生成 200 token 的请求也占着 512 MiB。vLLM 论文实测，预留 + 碎片让 KV 显存利用率只有 20%–40%。

8.2.2 分页

把 KV 切成固定大小的块（如 16 token），物理块散落在显存池里，每个请求一张 block table，记录“第 i 个逻辑块 → 第 j 个物理块”。生成新 token 时，只有当前块写满才申请新块；注意力 kernel 按 block table 取块。



手算：80 GB 卡，7B 权重 14 GB，留 60 GB 给 KV。每 token 128 KiB，总容量

$$60 \times 1024/0.125 = 491520 \text{ token}.$$

按 4096 预留，只能放

$$491520/4096 = 120 \text{ 条}.$$

如果真实平均上下文 512 token，按需分配能放

$$491520/512 = 960 \text{ 条}.$$

batch 上限放大 8 倍，Decode 每步权重读取摊到 8 倍的 token 上，吞吐近似同比例提升。这是 vLLM 2–4 倍吞吐提升的主要来源。

8.2.3 前缀共享与块大小

相同 system prompt 的多个请求可以共享前缀的物理块（引用计数），多轮对话、few-shot 模板受益明显。块大小是 trade-off：块越小碎片越少，但 block table 越长、kernel 寻址开销越大；16 是经验甜点。

8.3 量化：让权重和 KV 变小

8.3.1 对称 int8

$$s = \frac{\max|x|}{127}, \qquad q = \mathrm{round}(x/s), \qquad \hat{x} = qs. \tag{8.1}$$

本书 `py/quant.py` 对 7 个权重矩阵做 int8，保留 embedding/LayerNorm 为 float64（W8A16 的简化）：

```
=== per-tensor int8 quantization (weights only) ===
max |logit diff| = 0.36147, mean = 0.10664
top-1: base='r' quant='r' MATCH
top-5 overlap = 5/5

=== per-channel int8 quantization (weights only) ===
max |logit diff| = 0.20762, mean = 0.04851
top-1: base='r' quant='s' DIFF
top-5 overlap = 5/5
```

per-channel 的 logits 误差更小，但 top-1 反而变了：因为 `r` 和 `s` 原本只差 0.003，误差一小就会翻转。这说明**量化误差要看任务指标，不只看最大误差**；真实模型会做校准、混合精度、离群值处理（GPTQ/AWQ/SmoothQuant）。

8.3.2 W8A16 与 W4A16

方案	7B 权重	相对 bf16	适用
bf16	14 GB	1×	质量优先
W8A16	7 GB	1/2	主流线上
W4A16	3.5 GB	1/4	单卡大模型/边缘

激活保持 16 bit，因为激活是流经模型的中间量，误差逐层累积；权重只读一次，量化收益直接体现在带宽和显存上。Decode 是访存密集，权重减半近似 TPOT 减半。

8.4 FlashAttention：别让注意力矩阵落地

朴素注意力会把 $T \times T$ 的分数矩阵写回 HBM，再读出来做 softmax，再写回，再读出来乘 V。$T = 8192$ 时，这个矩阵有 6700 万个元素，读写量是天文数字。

FlashAttention 做两件事：

- **分块**：把 Q/K/V 切成小块，搬进 SRAM，在片上算完一块对输出的贡献，$T \times T$ 矩阵从不落地。
- **在线 softmax**：边看块边维护当前最大值和归一化分母，每来一个新块就修正已累计结果；全部看完后与一次性 softmax 严格相等。

HBM 访问量从 $O(T^2)$ 降到近似 $O(T)$（注意力计算量仍是 $O(T^2)$），显存从 $O(T^2)$ 降到 $O(T)$。这不是“更快的近似”，是“更少的搬运”，结果完全一致。

8.5 投机解码：小模型起草，大模型验收

Decode 的死结是串行：第 t 个 token 没出来，第 $t+1$ 个没法算。投机解码让一个小模型先连续起草 k 个 token，再让大模型一次前向并行验证这 k 个位置：

1. 小模型自回归生成 k 个候选；
2. 大模型对“前缀 $+\ k$ 个候选”做一次前向，得到 $k+1$ 个位置的分布；
3. 从第一个候选开始比对，接受直到第一个不匹配；不匹配的位置用大模型自己的 token 修正，后面的候选丢弃；
4. 本轮至少前进 1 个 token，最多前进 $k+1$ 个。

设平均接受 a 个，一次大模型前向产出 $a+1$ 个 token，加速比约

$$\frac{a+1}{1+kc}, \tag{8.2}$$

其中 c 是草稿模型相对大模型的前向成本。$k=4, a=3, c=1/50$ 时约 $4/1.08 \approx 3.7$ 倍。

关键性质：输出分布与大模型逐个生成完全一致；小模型只影响速度，不影响质量。接受率取决于草稿模型和大模型的一致性：代码补全、格式化输出接受率高，开放式创作接受率低。

8.6 PD 分离：推理集群的读写分离

Prefill 计算密集，Decode 访存密集；混在一个池子里，长 prompt 的 Prefill 会拖慢所有 Decode 的 TPOT。PD 分离把集群拆成两池：

- **Prefill 池**：跑 prompt 前向，产出首 token 和完整 KV；要算力，可以用更激进的 TP。
- **Decode 池**：接收 KV，进入连续批处理，逐 token 输出；要显存和带宽，可以用更大的 batch。
- **KV 传输**：Prefill 产出的 KV 通过网络送到 Decode 实例。7B、4k 上下文约 512 MiB；50 GB/s 链路上约 10 ms，NVLink 域内不到 1 ms。

这引入了分布式系统熟悉的问题：KV 传输失败重传或回源重算、Decode 实例挂了请求迁移、两池容量按 prompt/output 长度分布配比。它和数据库读写分离是同构的。

8.7 SLO 与容量规划

8.7.1 三个指标

- **TTFT**：请求到达到第一个 token 返回，约等于排队 + Prefill。
- **TPOT**：相邻输出 token 的平均间隔，约等于 Decode 一步的时间。
- **Goodput**：单位时间内满足 TTFT/TPOT SLO 的请求数；原始吞吐高但超时的请求不计入。

端到端延迟：

$$T_{\text{e2e}} = \text{TTFT} + (\text{输出 token 数} - 1) \times \text{TPOT}. \tag{8.3}$$

8.7.2 7B 手算

SLO：TTFT $\leq$ 500 ms，TPOT $\leq$ 50 ms，输出 200 token。理论端到端：

$$0.5 + 199 \times 0.05 = 10.45 \text{ s}.$$

单请求 Decode 理论 4.7 ms/token，看似余量很大；但 batch 32 时，每步要读 14 GB 权重 + 16 GB KV，约 10 ms，TPOT 仍在 50 ms 内。batch 再翻倍到 64，KV 读取约 32 GB，单步超过 15 ms，TPOT 逼近红线，goodput 开始下降。

8.7.3 容量规划的顺序

1. 算显存：权重 + KV + 框架开销，得到最大并发序列数。
2. 算带宽：每步权重 + KV 读取量，得到 TPOT 下限。
3. 算算力：Prefill 的 FLOPs，得到 TTFT 下限。
4. 算排队：按到达率、服务时间、SLO 反推需要多少副本。
5. 压测：用真实长度分布测 P99，不要用均值。

8.8 生产检查清单

工程视角

- 是否按 token 而不是按请求计费/限流?
- KV Cache 是否按需分配、能否前缀共享、是否有容量上限和淘汰策略?
- 是否支持连续批处理、chunked prefill、抢占/重算?
- 量化方案是否做过任务级评估（不只困惑度)?
- 是否有 TTFT/TPOT/goodput 的分位监控?
- 是否支持流式输出、断流重试、取消?
- PD 分离后 KV 传输失败/Decode 挂掉的恢复路径是什么?

- 模型/tokenizer/位置编码/KV 格式的版本是否绑定?

8.9 本章小结

- 连续批处理把 slot 利用率从 64.5% 提到 94.2%；PagedAttention 把 KV 利用率从 20%–40% 提到接近满载。
- 量化省显存和带宽，但要用任务指标评估；W8A16 是线上主流。
- FlashAttention 用分块 + 在线 softmax 消除 $T \times T$ 矩阵的 HBM 读写。
- 投机解码用接受率换串行延迟，输出分布不变。
- PD 分离让 Prefill 和 Decode 各自扩缩容，代价是 KV 传输和故障恢复。
- 容量规划的顺序是显存 → 带宽 → 算力 → 排队 → 压测。

第 9 章

一次参数更新：从 2 个参数开始

一句话

训练 = 前向算预测、损失量化误差、反向算梯度、优化器更新参数。这个循环在 2 个参数上和 70 亿个参数上完全同构；差别只是标量变成矩阵、手算变成自动微分、单卡变成集群。

9.1 一个只有 2 个参数的神经元

模型：

$$\hat{y} = wx + b. \tag{9.1}$$

初始 $w = 0.5, b = 0$；训练样本 $x = 2, y = 3$；损失用均方误差：

$$L = (\hat{y} - y)^2. \tag{9.2}$$

9.1.1 前向

$$\hat{y} = 0.5 \times 2 + 0 = 1, \qquad L = (1 - 3)^2 = 4.$$

预测 1，目标 3，损失 4。

9.1.2 反向

链式法则：

$$\frac{\partial L}{\partial \hat{y}} = 2(\hat{y} - y) = 2(1 - 3) = -4,$$

$$\frac{\partial \hat{y}}{\partial w} = x = 2, \qquad \frac{\partial \hat{y}}{\partial b} = 1,$$

$$\frac{\partial L}{\partial w} = -4 \times 2 = -8, \qquad \frac{\partial L}{\partial b} = -4 \times 1 = -4.$$

负号表示 “w 增大，损失减小”。梯度绝对值告诉我们责任大小：w 的梯度是 b 的两倍，因为 $x = 2$ 放大了它的影响。

9.1.3 更新

取学习率 $\eta = 0.05$：

$$w' = 0.5 - 0.05 \times (-8) = 0.9, \qquad b' = 0 - 0.05 \times (-4) = 0.2.$$

新预测 $0.9 \times 2 + 0.2 = 2.0$，新损失 $(2-3)^2 = 1$。一步从 4 降到 1。

9.1.4 十步之后

step	w	b	$\hat{y}$	L
0	0.500000	0.000000	1.000000	4.00000000
1	0.900000	0.200000	2.000000	1.00000000
2	1.100000	0.300000	2.500000	0.25000000
3	1.200000	0.350000	2.750000	0.06250000
4	1.250000	0.375000	2.875000	0.01562500
5	1.275000	0.387500	2.937500	0.00390625
6	1.287500	0.393750	2.968750	0.00097656
7	1.293750	0.396875	2.984375	0.00024414
8	1.296875	0.398438	2.992188	0.00006104
9	1.298437	0.399219	2.996094	0.00001526
10	1.299219	0.399609	2.998047	0.00000381

表 9.1: 2 参数神经元的梯度下降。损失每步约乘 0.25。

损失按 0.25 的因子几何衰减，因为目标是最小二乘，且步长合适。$w \to 1.3, b \to 0.4$，而 $1.3 \times 2 + 0.4 = 3$，正好拟合。

9.1.5 学习率的影响

- η 太小：每步只移动一点点，收敛慢，浪费算力。
- η 太大：一步跨过最小值，来回震荡甚至发散。
- 合适的 η：损失稳定下降；对这个小例子，0.05 刚好，0.1 会一步跳到目标（因为梯度大），再大就可能震荡。

9.2 从标量到矩阵：公式不变

把 x 换成向量、w 换成矩阵：

$$\mathbf{y} = \mathbf{x}W + \mathbf{b}. \tag{9.3}$$

损失对 W 的梯度：

$$\frac{\partial L}{\partial W} = \mathbf{x}^\top \frac{\partial L}{\partial \mathbf{y}}, \tag{9.4}$$

对 $\mathbf{b}$：

$$\frac{\partial L}{\partial \mathbf{b}} = \sum_{\text{batch}} \frac{\partial L}{\partial \mathbf{y}}, \tag{9.5}$$

对输入：

$$\frac{\partial L}{\partial \mathbf{x}} = \frac{\partial L}{\partial \mathbf{y}} W^{\top}. \tag{9.6}$$

这三条是 Transformer 反向传播里出现频率最高的公式；第 10 章会逐个算子推导。

9.3 自动微分：让框架替你走链式法则

手写梯度容易错，尤其到注意力。框架用自动微分：前向时记录计算图，反向时倒序调用每个算子的本地导数。本书的 Python 版在 `py/autograd.py`，核心是一个 `Tensor` 类和 `backward()`。

Listing 9.1: 自动微分的反向遍历（`py/autograd.py` 节选）

```python
def backward(loss):
    topo, seen = [], set()
    def build(t):
        if id(t) in seen: return
        seen.add(id(t))
        for p in t._prev: build(p)
        topo.append(t)
    build(loss)
    loss.grad = [1.0]
    for t in reversed(topo):
        t._backward()
```

以乘法 $c = ab$ 为例，反向规则是：

$$\frac{\partial L}{\partial a} = \frac{\partial L}{\partial c} \cdot b, \qquad \frac{\partial L}{\partial b} = \frac{\partial L}{\partial c} \cdot a.$$

代码里就是两行累加。所有算子（加法、矩阵乘、softmax、LayerNorm、交叉熵）都按这个模式写，最后拼成完整的 Transformer。

9.3.1 数值梯度检查

自动微分写完，必须对拍。取 $w = 0.5, b = 0$，用中心差分：

$$\frac{\partial L}{\partial w} \approx \frac{L(w + 10^{-6}) - L(w - 10^{-6})}{2 \times 10^{-6}} = -8.00000000,$$

和解析梯度 -8 一致。本书的 Go 训练器带一个 `TestGradients`，对 embedding、QKV、LayerNorm、MLP、输出层的若干元素做同样的检查，全部通过。

9.4 Go 版：把梯度手写出来

Python 版用自动微分，Go 版没有运算符重载，也没有现成的 autograd；作者把每个梯度显式写出来。这不是为了“更难”，而是为了证明：反向传播不是魔法，就是一堆矩阵乘。

以 LayerNorm 的反向为例（`go/tiny/train.go`）：

Listing 9.2: LayerNorm 反向（go/tiny/train.go 节选）

```go
for r := 0; r < x.Rows; r++ {
    // xhat = (x - mean) / sqrt(var + eps)
    // dxhat = dy * gamma
    // dx = inv/D * (D*dxhat - sum(dxhat) - xhat*sum(dxhat*xhat))
    for i := 0; i < cols; i++ {
        dx.Set(r, i, inv/n*(n*dxhat[i]-s1-xhat[i]*s2))
        dg[i] += dy.At(r, i) * xhat[i]
        db[i] += dy.At(r, i)
    }
}
```

这段代码和论文里的公式一一对应；如果符号错了，数值梯度检查会立刻发现。

9.5 为什么需要非线性

如果两层都是线性的：

$$\mathbf{y} = W_2(W_1\mathbf{x}) = (W_2W_1)\mathbf{x} = W\mathbf{x},$$

两层等价于一层，深度没有意义。非线性（ReLU/GELU/SwiGLU）让网络能拟合“异或”这种线性不可分函数。

手算

异或（XOR）：输入 $(0,0)\to 0$，$(0,1)\to 1$，$(1,0)\to 1$，$(1,1)\to 0$。一条直线无法把 0 和 1 分开；但一个隐藏层 + ReLU 可以：

$$h_1 = \mathrm{ReLU}(x_1 + x_2), \quad h_2 = \mathrm{ReLU}(x_1 + x_2 - 1),$$

$$y = h_1 - 2h_2.$$

验证：$(0,0)\to 0$，$(1,0)\to 1-0=1$，$(1,1)\to 2-2=0$。非线性制造了“折点”，才能拟合 XOR。

Transformer 的 MLP 就是“线性 → 非线性 → 线性”，注意力本身也含 softmax 非线性。没有这些，堆多少层都只是一个线性变换。

9.6 从 2 个参数到 6472 个参数

本书的 Transformer 有 6472 个参数（表 4.2）。训练时：

1. 前向：token → embedding → 注意力 → MLP → logits；
2. 损失：每个位置的交叉熵，取平均；
3. 反向：从损失出发，依次经过输出层、LayerNorm、MLP、注意力、embedding，算出每个参数的梯度；
4. 更新：AdamW，学习率 warmup + cosine。

这和 2 参数神经元是同一个循环，只是“标量乘法”变成了“矩阵乘法”，“一个梯度”变成了 6472 个梯度。

9.7 训练循环的骨架

Listing 9.3: 训练循环（py/train.py 节选）

```
for step in range(steps):
    lr = lr_at(step, steps, peak_lr)
    xs, ys = sample_batch(train_ids, block, batch, rng)
    total = None
    for x, y in zip(xs, ys):
        l = loss_fn(params, x, y, cfg)
        total = l if total is None else add(total, l)
    total = scale(total, 1.0 / len(xs))
    backward(total)
    clip_grads(params, 1.0)
    adam_step(params, state, step + 1, lr)
```

六个动作：采样、前向、损失、反向、裁剪、更新。真实训练还多了数据加载、混合精度、分布式通信、checkpoint，但这六步是核心。

9.8 本章小结

- 2 参数神经元走完前向/损失/反向/更新，损失从 4 降到 3.8×10^{-6}。
- 矩阵版的梯度公式：$dW = x^\top dy$，$db = \sum dy$，$dx = dyW^\top$。
- 自动微分 = 计算图 + 倒序链式法则；数值梯度是最好的对拍工具。
- Go 版手写每个梯度，LayerNorm 等公式与论文一一对应。
- 非线性让深度有意义；没有它，多层线性等于一层。
- 6472 参数 Transformer 的训练循环和 2 参数神经元同构。

第 10 章

语言模型训练：交叉熵与反向传播

一句话

语言模型的训练数据不需要人工标注：把一段文本错一位，前 T 个 token 是输入，后 T 个 token 是标签。损失是每个位置“正确下一个 token”的负对数概率；反向传播把误差分摊到 6472 个参数上。

10.1 数据：把文本错一位

取一段长度为 $T+1$ 的 token：

$$x_0, x_1, \ldots, x_T.$$

输入是 $x_0, \ldots, x_{T-1}$，标签是 $x_1, \ldots, x_T$：

$$\text{input} = [x_0, x_1, \ldots, x_{T-1}], \quad \text{target} = [x_1, x_2, \ldots, x_T].$$

模型在位置 t 看到 $x_0, \ldots, x_t$，预测 x_{t+1}。一本书、一个网页、一段代码，都可以这样切成训练样本，不需要标注。

手算

语料 `"the cat sat"` 的字符 ID 假设是 $[14, 7, 5, 0, 3, 2, 14]$。取前 6 个作为输入：

$$[14, 7, 5, 0, 3, 2],$$

标签是：

$$[7, 5, 0, 3, 2, 14].$$

位置 0 看到 `t`，要预测 `h`；位置 5 看到 `the ca`，要预测 `t`。

本书用随机窗口采样：每次从训练集随机取一个起点，切 $T = 32$ 个 token。一个 epoch 不是“把语料读一遍”，而是“采样若干 batch”；真实训练通常只跑一个“大 epoch”，因为语料有数万亿 token。

10.2 损失：交叉熵

模型在位置 t 输出 logits $\mathbf{z}_t \in \mathbb{R}^V$，softmax 后得到 p_t。真实标签是 y_t。单条序列的损失：

$$L = -\frac{1}{T}\sum_{t=0}^{T-1} \ln p_{t,y_t}. \tag{10.1}$$

本书 $V = 16$，随机猜时 $p = 1/16$，$L = \ln 16 = 2.773$。训练开始时 loss 约 3.5–4.0（初始化不好），收敛后约 0.2。

10.2.1 softmax + 交叉熵的梯度

这是全书最值得手推的公式。设一个位置的 logits 为 z_i，softmax 为 p_i，标签为 y。损失 $L = -\ln p_y$。

先算 p_i 对 z_j 的导数：

$$\frac{\partial p_i}{\partial z_j} = p_i(\mathbf{1}[i=j] - p_j). \tag{10.2}$$

于是

$$\frac{\partial L}{\partial z_j} = -\frac{1}{p_y}\frac{\partial p_y}{\partial z_j} = -\frac{1}{p_y}p_y(\mathbf{1}[y=j] - p_j) = p_j - \mathbf{1}[y=j]. \tag{10.3}$$

写成向量：

$$\frac{\partial L}{\partial \mathbf{z}} = \mathbf{p} - \text{onehot}(y). \tag{10.4}$$

预测概率与标签的差，就是 logits 的梯度。若 $p_y = 0.9$，梯度在正确类上是 -0.1；若 $p_y = 0.01$，梯度是 -0.99，纠错力度大得多。这就是交叉熵比 MSE 适合分类的原因。

对 T 个位置取平均，梯度再除以 T：

$$\frac{\partial L}{\partial \mathbf{z}_t} = \frac{1}{T}(\mathbf{p}_t - \text{onehot}(y_t)). \tag{10.5}$$

10.3 反向传播：逐个算子

下面按前向顺序列出每个算子的反向公式。所有矩阵都按行存放：$X \in \mathbb{R}^{T\times D}$，$W \in \mathbb{R}^{D\times D'}$。

10.3.1 线性层

前向：

$$Y = XW + \mathbf{1}\mathbf{b}^\top. \tag{10.6}$$

反向：

$$\frac{\partial L}{\partial W} = X^\top \frac{\partial L}{\partial Y}, \qquad \frac{\partial L}{\partial \mathbf{b}} = \sum_t \frac{\partial L}{\partial Y_{t,:}}, \qquad \frac{\partial L}{\partial X} = \frac{\partial L}{\partial Y}W^\top. \tag{10.7}$$

形状：$X^\top$ 是 $D \times T$，dY 是 $T \times D'$，乘出 $D \times D'$，和 W 同形。

10.3.2 残差

前向 $Y = X + F(X)$。反向：

$$\frac{\partial L}{\partial X} = \frac{\partial L}{\partial Y} + \frac{\partial L}{\partial Y}\frac{\partial F}{\partial X}. \tag{10.8}$$

代码里就是“梯度直接复制一份给残差分支”，这也是残差让梯度畅通的原因。

10.3.3 LayerNorm

前向：

$$\hat{x}_i = \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}}, \qquad y_i = \gamma_i \hat{x}_i + \beta_i. \tag{10.9}$$

反向（对一行）：

$$\frac{\partial L}{\partial \hat{x}_i} = \frac{\partial L}{\partial y_i}\gamma_i, \tag{10.10}$$

$$\frac{\partial L}{\partial x_i} = \frac{1}{D\sqrt{\sigma^2 + \epsilon}}\left(D\frac{\partial L}{\partial \hat{x}_i} - \sum_j \frac{\partial L}{\partial \hat{x}_j} - \hat{x}_i \sum_j \frac{\partial L}{\partial \hat{x}_j}\hat{x}_j\right). \tag{10.11}$$

γ, β 的梯度是 $\sum_t dy_t \hat{x}_t$ 和 $\sum_t dy_t$。

10.3.4 ReLU

前向 $y = \max(0, x)$。反向：

$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial y} \cdot \mathbf{1}[x > 0]. \tag{10.12}$$

负数的梯度直接截断为 0。

10.3.5 注意力

前向：

$$S = \frac{QK^\top}{\sqrt{D}} + M, \quad A = \mathrm{softmax}(S), \quad O = AV. \tag{10.13}$$

反向：

$$\frac{\partial L}{\partial V} = A^\top \frac{\partial L}{\partial O}, \tag{10.14}$$

$$\frac{\partial L}{\partial A} = \frac{\partial L}{\partial O}V^\top, \tag{10.15}$$

$$\frac{\partial L}{\partial S} = A \odot \left(\frac{\partial L}{\partial A} - \left(\sum_j A_{ij}\frac{\partial L}{\partial A_{ij}}\right)\mathbf{1}^\top\right), \tag{10.16}$$

$$\frac{\partial L}{\partial Q} = \frac{1}{\sqrt{D}}\frac{\partial L}{\partial S}K, \tag{10.17}$$

$$\frac{\partial L}{\partial K} = \frac{1}{\sqrt{D}}\left(\frac{\partial L}{\partial S}\right)^\top Q. \tag{10.18}$$

掩码位置的反向梯度为 0。softmax 的反向是“A 乘（上游梯度减去加权平均）”，保证每行梯度和为 0。

10.3.6 Embedding

前向是查表：$X_t = E[\mathrm{id}_t]$。反向是“散射加”：

$$\frac{\partial L}{\partial E[i]} = \sum_{t:\mathrm{id}_t=i} \frac{\partial L}{\partial X_t}. \tag{10.19}$$

同一个 token 在序列里出现多次，梯度要累加。

10.3.7 输出层

$Z = H_f W_{\mathrm{lm}} + \mathbf{b}$，然后接交叉熵。梯度从 $dZ = (P - Y)/T$ 开始，按线性层公式反传。

10.4 优化器：AdamW

裸 SGD 对每个参数用同一个学习率，遇到不同尺度的梯度会很难调。Adam 维护一阶矩和二阶矩：

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t, \tag{10.20}$$

$$v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2, \tag{10.21}$$

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t}, \qquad \hat{v}_t = \frac{v_t}{1-\beta_2^t}, \tag{10.22}$$

$$\theta_t = \theta_{t-1} - \eta \left(\frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} + \lambda \theta_{t-1} \right). \tag{10.23}$$

λ 是权重衰减（weight decay），最后一项是 AdamW 的“解耦权重衰减”。

10.4.1 梯度裁剪

把所有梯度拼成一个大向量，算全局范数：

$$\|g\| = \sqrt{\sum_i g_i^2}. \tag{10.24}$$

如果 $\|g\| > c$，所有梯度乘以 $c/\|g\|$。它防止单个异常 batch 把参数甩飞，是 loss spike 的第一道防线。

10.4.2 学习率调度

本书用 warmup + cosine：

$$\eta(t) = \begin{cases} \eta_{\max} \dfrac{t}{T_w}, & t < T_w, \\ 0.1\eta_{\max} + 0.9\eta_{\max} \dfrac{1 + \cos(\pi \frac{t - T_w}{T - T_w})}{2}, & t \geq T_w. \end{cases} \tag{10.25}$$

前 20 步线性升温，之后余弦降到峰值的 10%。warmup 让 Adam 的动量估计先稳定；cosine 让末期小步精调。

10.5 Python 训练：真实输出

```
vocab=16 params=6472 block=32 train_chars=547 val_chars=97
 step       lr     loss    gnorm      ms
    0  0.00015   3.5833   10.403   379.6
   20  0.00300   1.9335    3.419  3642.6
   40  0.00299   1.3220    4.106  6857.6
...
  300  0.00172   0.3322    1.911 48817.6
...
  599  0.00030   0.2077    2.040 97585.8  val=0.2934
sample: the cat ran to the log .
final_loss: 0.2077, val_loss: 0.2973, elapsed: 99.8 s
```

600 步、batch 4、每步 4 条 32 token 序列，共约 $600 \times 4 \times 32 = 76800$ 个 token 的预测。损失从 3.58 降到 0.208，验证损失 0.297，没有明显过拟合（语料太小，过拟合也不奇怪）。

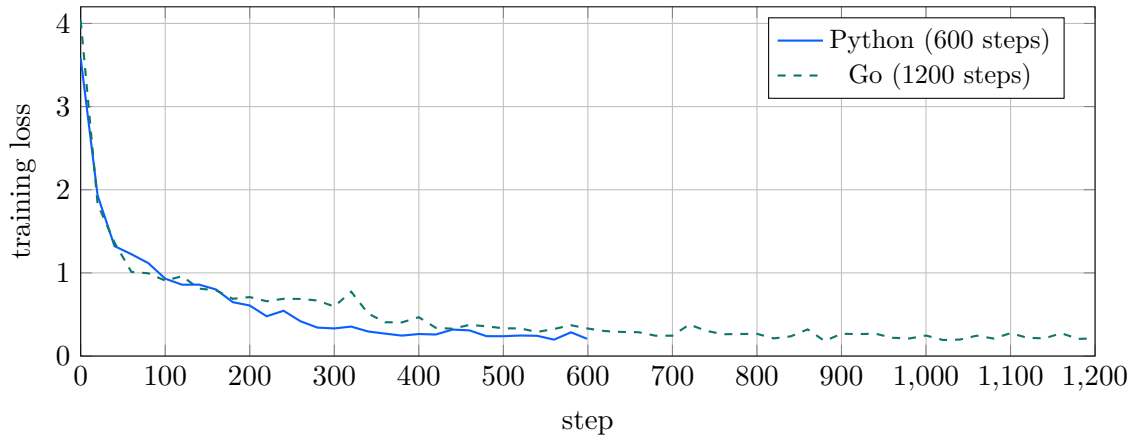


图 10.1: 两条损失曲线。Python 600 步到 0.208；Go 1200 步到 0.212。横轴步数不同，但形状一致：先快速下降，后缓慢收敛。

10.6 Go 训练：手写反向传播

Go 版用同样的结构，但反向传播是显式写的。训练 1200 步、batch 4：

```
vocab=16 params=6472 block=32 train_chars=547 val_chars=97
 step       lr     loss    gnorm      ms
    0  0.00015   4.0360   23.644     3.0
   20  0.00300   1.8303    2.974    43.0
   40  0.00300   1.3644    4.649    72.0
...
 1199  0.00030   0.2125    2.325  1572.0
sample: the cat . the saw the cat .
final_loss: 0.2125, val_loss: 0.2571, elapsed: 1.76 s
```

Go 每步约 1.5 ms，Python 每步约 165 ms，差约 110 倍。两者损失曲线形状一致，说明手写反向传播是对的；数值不完全相同，因为初始化、随机数、浮点归约顺序不同。

指标	Python	Go
参数量	6472	6472
步数	600	1200
每步耗时	约 165 ms	约 1.5 ms
最终训练损失	0.208	0.212
验证损失	0.297	0.257
样本	`the cat ran to the log .`	`the cat . the saw the cat .`

表 10.1: Python 与 Go 的训练对比。Go 的样本略差，是因为它只训练了 1200 步、初始化不同；两者都在同一量级。

10.7 调试：loss 不降怎么办

1. **先看标签错位**。输入和标签必须错一位；如果标签等于输入，模型会学会复制，loss 也会降，但推理是废的。
2. **看初始 loss**。随机猜是 $\ln V$；如果初始 loss 远高于它，初始化或数据有 bug；远低于它，可能泄漏了标签。
3. **做梯度检查**。取几个参数，用中心差分对拍；误差大于 10^{-4} 就要查反向公式。
4. **调小学习率**。 loss 震荡/飞掉，先把 lr 降 10 倍；如果还飞，查 NaN/Inf。
5. **查数据**。空 batch、全 padding、标签越界、mask 反了，都会让 loss 不降。
6. **看验证集**。训练 loss 降、验证 loss 升，是过拟合；两个都不降，是优化/数据问题。

10.8 从训练到推理

训练结束后，`save_weights` 把配置、词表、每个参数的形状和数据写进 JSON。推理脚本 `load_weights` 读回，构造同样的模型，加载权重，就可以生成。本书的 Python 训练产物可以被 Go 推理直接读取，因为 JSON 格式和参数命名一致；这是跨语言对拍的基础。

```
save_weights("weights/tiny_py.json", cfg, vocab, params)
cfg, vocab, params = load_weights("weights/tiny_py.json")
```

真实训练的 checkpoint 还包含优化器状态、学习率调度器、epoch/step、随机数状态、数据管道位置；少一项，断点恢复就可能不一致。

10.9 本章小结

- 语言模型训练数据 = 文本错一位；标签不需要人工标注。
- 交叉熵的 logits 梯度是 $p -$ one-hot，干净且数值稳定。
- 反向传播逐个算子：线性、残差、LayerNorm、ReLU、注意力、embedding。
- AdamW + 梯度裁剪 + warmup/cosine 是标准组合。

- Python 600 步 loss 0.208，Go 1200 步 loss 0.212；手写反向传播与自动微分一致。
- 调试顺序：标签、初始 loss、梯度检查、学习率、数据、验证集。

第 11 章

从 6472 到 7B：算力、显存、时间

一句话

大模型训练的两本账：算力账 $6ND$（N 参数、D token），显存账每参数 16 字节（fp32 主权重、梯度、两个动量）。这两条公式能回答“要几张卡、跑多久、花多少钱”。

11.1 参数量：从结构到数字

11.1.1 公式

对一个 decoder-only Transformer，设层数 L、隐藏维度 D、FFN 维度 D_{ff}、词表 V。每层参数大致为：

- 注意力：W_q, W_k, W_v, W_o 各 D^2，加偏置 $4D$，共 $4D^2 + 4D$；
- MLP：若用 ReLU 两层，$2DD_{ff} + D + D_{ff}$；若用 SwiGLU 三门，$3DD_{ff} + D + D_{ff}$；
- 两个 LayerNorm：$4D$。

总参数：

$$N \approx L\left(4D^2 + \alpha DD_{ff} + 8D\right) + 2VD, \tag{11.1}$$

其中 $\alpha = 2$（ReLU）或 3（SwiGLU）；$2VD$ 是输入 embedding 和输出投影（若权重绑定则只有 VD）。

11.1.2 7B 算例

取 $L = 32, D = 4096, D_{ff} = 11008, V = 32000$，SwiGLU（$\alpha = 3$）：

$$4D^2 = 67.1\text{M}, \quad 3DD_{ff} = 135.3\text{M},$$

每层约 202.4M，32 层约 6.48B；embedding 和输出各 131M，合计约 6.74B。这就是“7B”的来源。

部分	每层/每项	总计
注意力 $4D^2$	67.1M	2.15B
MLP $3DD_{ff}$	135.3M	4.33B
LayerNorm 等	0.016M	0.0005B
embedding + 输出	262.1M	0.26B
合计		6.74B

表 11.1: 7B 模型的参数账本。不同模型的具体数字略有差异，量级一致。

11.2 算力：$6ND$ 的来历

11.2.1 矩阵乘的 FLOPs

一个 $n \times m$ 乘 $m \times p$ 的矩阵乘，有 np 个输出元素，每个元素做 m 次乘法和 $m-1$ 次加法，FLOPs 约 $2nmp$。若把权重矩阵 $W \in \mathbb{R}^{m\times p}$ 看成 mp 个参数，则每个参数在一次前向中参与一次乘法、一次加法，共 2 FLOPs。

11.2.2 前向 $2N$

每个 token 流过模型，所有参数各参与一次乘加，前向约 $2N$ FLOPs/token。严格说还有注意力分数、softmax、LayerNorm，但它们相对 $2N$ 是小项；长上下文时注意力项会变大（见 11.6）。

11.2.3 反向 $4N$

反向对每个参数要算两个梯度：

- 梯度对输入的偏导（把误差传给上一层）：约 $2N$；
- 梯度对参数的偏导（累加 dW）：约 $2N$。

合计 $4N$ FLOPs/token。直观理解：反向要同时给“下一层”和“参数”两个对象交账，所以是前向的两倍。

11.2.4 合计 $6ND$

每个 token 前向 + 反向 $= 6N$；训练 D 个 token：

$$\text{总 FLOPs} \approx 6ND. \tag{11.2}$$

手算

7B（$N = 7\times 10^9$）、2T token（$D = 2\times 10^{12}$）：

$$6ND = 6\times 7\times 10^9 \times 2\times 10^{12} = 8.4\times 10^{22}\ \text{FLOPs}.$$

H100 级 bf16 峰值约 10^{15} FLOPS；MFU 50% 时有效算力 5×10^{14}。单卡：

$$\frac{8.4\times 10^{22}}{5\times 10^{14}} = 1.68\times 10^8\ \text{s} \approx 5.3\ \text{年}.$$

1024 卡：

$$\frac{1.68 \times 10^8}{1024} = 1.64 \times 10^5 \text{ s} \approx 1.9 \text{ 天}.$$

11.2.5 70B 算例

70B、15T token：

$$6 \times 70 \times 10^9 \times 15 \times 10^{12} = 6.3 \times 10^{24} \text{ FLOPs}.$$

4096 张 H100、MFU 50%：

$$\frac{6.3 \times 10^{24}}{4096 \times 5 \times 10^{14}} \approx 3.1 \times 10^6 \text{ s} \approx 36 \text{ 天}.$$

实际还要乘上故障、重启、数据加载、通信等待，往往 1.5–2 倍。

11.3 显存：为什么 7B 训练要 100 GB 以上

11.3.1 每参数 16 字节

混合精度训练（bf16 计算 + fp32 主权重）：

状态	每参数字节	说明
fp32 主权重	4	更新用
fp32 梯度	4	反向产生
Adam 一阶动量 m	4	fp32
Adam 二阶动量 v	4	fp32
合计	16	不含 bf16 权重和激活

7B：$7 \times 10^9 \times 16 = 112$ GB。再加上 bf16 权重 14 GB、激活值、通信缓冲、碎片，单卡 80 GB 肯定放不下。

11.3.2 激活值

激活值是前向每层的输出，反向要用。粗略估算：

$$\text{激活显存} \approx B \times T \times L \times D \times k, \tag{11.3}$$

其中 k 是每层每 token 保存的中间量倍数（通常 10–20，取决于实现和是否重算）。$B = 8, T = 4096, L = 32, D = 4096$ 时，仅 $BTD = 1.3 \times 10^8$ 个数，fp16 就 268 MB；乘 $k = 10$ 约 2.7 GB。实际更大，因为注意力、MLP 中间量都要存。

梯度检查点（activation checkpointing）只存每层的输入，反向时重算中间量，用约 1/3 的额外计算换 3–5 倍的激活显存。第 13 章展开。

11.3.3 推理的账本

推理不存梯度、不存优化器，只存 bf16 权重 14 GB + KV Cache。这就是“14 GB 能放下、112 GB 放不下”的落差，也是第 4 章并行技术存在的根本原因。

11.4 时间与成本

11.4.1 时间公式

$$T_{\text{train}} = \frac{6ND}{N_{\text{gpu}} \times \text{peak} \times \text{MFU}}. \tag{11.4}$$

它把“模型多大、数据多少、卡多少、效率多高”四个变量放在一个式子里。优化训练，就是优化这四个变量。

11.4.2 成本

按 \$2/GPU-hour：

- 7B、2T、1024 卡、1.9 天：$1024 \times 1.9 \times 24 \times 2 \approx \$93k$；
- 70B、15T、4096 卡、36 天：$4096 \times 36 \times 24 \times 2 \approx \$7.1M$；
- 加上失败重跑、存储、网络、人力，实际可能翻倍。

这就是为什么“预训练烧钱”：大部分算力花在 95%–99% 的预训练阶段，SFT/对齐只占零头。

11.5 Scaling Law：参数、数据、算力怎么配

经验上，损失随参数量 N、数据量 D 近似幂律下降：

$$L(N, D) \approx L_\infty + aN^{-\alpha} + bD^{-\beta}. \tag{11.5}$$

在固定算力 $C \approx 6ND$ 下，最优分配大致满足 $N \propto C^{0.5}, D \propto C^{0.5}$。Chinchilla 的经验是：

$$D_{\text{opt}} \approx 20N. \tag{11.6}$$

7B 模型约需 140B token 达到“算力最优”。但现代模型常常训练 2T 甚至更多 token，因为**推理成本**远大于训练成本：多花训练算力换一个更小、更便宜的推理模型是划算的。这就是“过度训练”（overtraining）的工程逻辑。

11.6 玩具模型与 7B 的对照

11.7 $6ND$ 的误差来源

- **注意力项**：$O(T^2D)$ 每层，T 很大时不可忽略；$T = 4096, D = 4096$ 时，注意力约占总计算的 10%–20%。

指标	本书玩具	7B 算例
层数 L	1	32
隐藏维度 D	24	4096
FFN 维度	48	11008
头数	1	32
词表 V	16	32000
上下文 T	32	4096
参数量 N	6472	6.74B
训练 token D	76,800	2T
训练 FLOPs $6ND$	3.0×10^9	8.1×10^{22}
优化器状态	0.1 MB	108 GB
权重（bf16）	13 KB	13.5 GB
单卡训练时间	98 s（CPU）	5.3 年（H100）

表 11.2: 同一个公式，两个尺度。玩具模型的数字都可以在日志里找到。

- **Embedding/输出层**：词表大时，输出投影的 FLOPs 占比上升。
- **激活重算**：梯度检查点会增加约 1/3 前向计算。
- **通信/等待**：AllReduce、流水线气泡、数据加载都不产生有效 FLOPs，体现在 MFU 里。
- **稀疏/量化**：混合精度、FP8、稀疏注意力会改变实际峰值。

所以 $6ND$ 是“量级估算”，不是精确账单；做容量规划时要留 20%–30% 余量。

11.8 本章小结

- 参数量由层数、维度、FFN、词表决定；7B 约 6.74B。
- 前向 $2N$、反向 $4N$、合计 $6ND$；7B/2T 约 8.4×10^{22} FLOPs。
- 训练每参数 16 字节，7B 约 112 GB；推理只需 bf16 权重 14 GB + KV。
- 时间 $= 6ND/(N_{\text{gpu}} \times \text{peak} \times \text{MFU})$；7B/2T/1024 卡约 1.9 天。
- Chinchilla 的 $D \approx 20N$ 是算力最优；现代模型为了推理成本会过度训练。
- $6ND$ 忽略注意力、重算、通信，做预算要留余量。

第 12 章

真实训练流水线：从网页到可用模型

一句话

预训练只占流程的一半：前面有 PB 级数据清洗/去重/配比，后面有 SFT、偏好对齐、评测、部署。算力的大头在预训练，效果的大头往往在数据和对齐。

12.1 全景

阶段	输入	产出/目标
数据工程	网页、书、代码、论文	干净、去重、配比合理的 token 流
预训练	数万亿 token	下一词预测，学语言与世界知识
SFT	指令-回答对	学会按指令回答，loss 只算回答
对齐	偏好对比	更有用、更无害；RLHF 或 DPO
评测/部署	checkpoint	基准、红队、推理服务

算力占比通常是预训练 95%–99%，SFT/对齐 1%–5%。但 SFT/对齐的数据质量和工程细节，直接决定模型“好不好用”。

12.2 数据工程

12.2.1 清洗

原始网页是 HTML：导航、广告、脚本、页脚都要去掉。保留正文、代码缩进、公式、表格结构。清洗规则要版本化、可回放，因为训练中途发现脏数据时要能重跑局部管道。

12.2.2 去重

精确去重只能抓一字不差的复制；近似重复（转载、镜像、改几个字）用 MinHash。把文档切成 shingles，做 k 次哈希取最小值，得到 k 个数的签名；相似文档的签名大量撞车。把签名分成 band，只要一个 band 全撞就判为候选重复。

手算

两篇文档 Jaccard 相似度 0.9；一个 band 有 4 个位置，全撞概率 $0.9^4 = 0.656$；32 个 band 至少一个撞的概率 $1 - (1 - 0.656)^{32} \approx 1$。相似度 0.5 时，全撞概率 $0.5^4 = 0.0625$，漏网概率

$(1-0.0625)^{32} \approx 0.127$。阈值是误杀/漏网的权衡。

12.2.3 质量过滤与配比

启发式规则（长度、标点、乱码率、敏感词）加一个小分类器，过滤低质文本。配比（网页/书/代码/数学/多语言）没有理论最优，只有实验最优；头部团队把配比当核心机密。小模型（1B）扫配比，再放大到大模型，是常见做法。

12.2.4 污染

测试集的题、答案、改写版本如果出现在训练集，benchmark 会虚高。要做 n-gram 重叠检测、哈希黑名单、时间切分；污染是“看不见的过拟合”。

12.3 Tokenizer 与 Packing

12.3.1 词表

词表大小影响 embedding 参数、序列长度、推理成本。英文为主可以用 32k–50k；多语言/代码需要更大词表。词表要在目标分布上训练，不能拿英文词表切中文。

12.3.2 Packing

训练序列固定长度 T（如 4096）。真实文档长短不一；不 packing 时，短文档要 padding，padding 位置照样参与注意力/FFN 计算，浪费算力。

手算

平均文档 1024 token，$T = 4096$。不 packing 时有效 token 占比 $1024/4096 = 25\%$，75% 算力在白算。Packing 把多篇文档首尾相接，用 EOS 分隔，配合块对角注意力掩码阻止跨文档注意，有效占比接近 100%，吞吐约 4 倍。

Packing 的坑：mask 边界要和文档边界一起拼；拼错一格，损失就算到别的文档上。这类 bug 不报错、不崩溃，只是模型悄悄学歪。

12.4 预训练

12.4.1 目标与监控

目标是下一词预测，损失是交叉熵。训练数周，唯一仪表盘是 loss 曲线：

- 初始 loss 应接近 $\ln V$；远高/远低都是 bug。
- 健康曲线：快速下降，后缓慢幂律下降，有微小毛刺。
- 异常：毛刺变密（脏数据/梯度异常）、突跳（lr 太高/坏卡）、NaN（数值溢出）。

12.4.2 Checkpoint

训练状态不只是权重：还有优化器 m/v、学习率调度器、step、随机数状态、数据管道位置。少一项，断点恢复就可能不一致。写 checkpoint 要原子（临时文件 + rename）、校验和、保留最近 N 个；否则一次写坏，几天白跑。

12.5 SFT：只在回答上算损失

预训练模型是“续写机器”，不是“助手”。SFT 用指令-回答对，把问题和回答拼成一条序列，但只对回答 token 算损失；问题部分的标签设为 -100（PyTorch 的 ignore index）。

位置	token	label	log p	计入损失
0	<s>	-100	-0.10	否
1	1	-100	-0.20	否
2	+	-100	-0.30	否
3	1	-100	-0.10	否
4	=	-100	-0.20	否
5	?	-100	-0.40	否
6	2	2	-0.05	是
7	</s>	</s>	-0.02	是

表 12.1: SFT 的 loss mask。只有最后两个 token 计入损失。

手算

所有位置平均：$-(0.1 + 0.2 + 0.3 + 0.1 + 0.2 + 0.4 + 0.05 + 0.02)/8 = 0.1713$。只算回答：$-(0.05 + 0.02)/2 = 0.0350$。问题部分的损失被 mask 掉，模型不会浪费容量去“复述问题”。

本书的 `py/sft_dpo.py` 打印了这张表和两个损失。真实 SFT 还要处理多轮对话、系统提示、工具调用格式、packing 边界；mask 错一格，模型就会学到“把问题再写一遍”。

12.6 对齐：RLHF 与 DPO

12.6.1 RLHF/PPO 为什么贵

PPO 阶段显存里同时有四个模型：

- policy（正在训练）；
- reference（冻结，防跑偏）；
- reward model（打分）；
- value/critic（估计优势）。

7B bf16 每个约 14 GB，四个权重 56 GB；policy/critic 还要优化器状态。加上在线采样（policy 生成、RM 打分、再更新），是“训练套着推理”的混合负载，吞吐被生成拖住。

12.6.2 DPO

DPO 把“奖励模型 + 在线 RL”折叠成一个监督损失，只需要 policy 和 reference：

$$L_{\text{DPO}} = -\ln \sigma \left(\beta \left[(\ln p_\theta(y_w|x) - \ln p_\theta(y_l|x)) - (\ln p_{\text{ref}}(y_w|x) - \ln p_{\text{ref}}(y_l|x)) \right] \right). \tag{12.1}$$

y_w 是偏好回答，y_l 是拒绝回答，β 控制偏离 reference 的强度。

β	margin	loss
0.1	0.0300	0.6783
0.5	0.1500	0.6210
1.0	0.3000	0.5544
2.0	0.6000	0.4375

表 12.2: DPO 损失随 β 的变化（`py/sft_dpo.py` 输出）。margin 越大，损失越小。

DPO 的工程优点是“像 SFT 一样训练”：两个模型、无在线采样、无 RM 服务；代价是效果上限略逊于调好的 PPO，对偏好数据分布更敏感。

12.7 评测

- **基准**：MMLU、GSM8K、HumanEval、C-Eval 等；注意污染和 prompt 格式。
- **held-out**：从训练分布留出，测泛化；但真实用户分布往往不同。
- **LLM-as-judge**：用强模型打分，便宜但有偏差；要和人工评估校准。
- **红队**：安全、越狱、隐私、偏见；上线前必须做。
- **在线 A/B**：最终指标是用户满意度/任务完成率，不是 benchmark 分数。

12.8 本地 mini pipeline

本书的 demo 把“预训练 + 推理”跑通了，但没有真正的 SFT/RLHF 训练，因为那需要一个预训练底座和指令数据。你可以用下面的顺序体验完整链路：

```
# 1. 数据：字符级语料
cat data/corpus.txt

# 2. 训练：下一词预测
python3 py/train.py --steps 600 --batch 4 --out weights/tiny_py.json

# 3. 评测：验证损失 + 生成样本
python3 py/infer.py --temperature 0

# 4. SFT/DPO 的数学：mask 与偏好损失
python3 py/sft_dpo.py
```

真实 SFT 的最小改动是：把 `loss_fn` 换成“只对 label $\neq -100$ 的位置算交叉熵”，数据换成指令-回答对；模型结构不变。

12.9 失败 playbook

现象	可能原因	处置
loss spike	lr 太高、脏 batch、坏卡	回滚 checkpoint、降 lr、跳过数据、换卡
NaN	fp16 溢出、log(0)、梯度爆炸	bf16、安全 softmax、裁剪、查异常 batch
OOM	batch/seq 太大、激活太多	减 batch、梯度检查点、ZeRO
数据慢	实时 tokenize、无预取	离线 tokenize/packing、预取、缓存
checkpoint 坏	非原子写、磁盘满	临时文件 +rename、校验和、保留多份
loss 不降	标签错位、mask 反、lr 太小	查数据、梯度检查、调 lr
评测虚高	测试集污染	n-gram 去重、时间切分、黑名单

12.10 本章小结

- 数据决定上限：清洗、去重、质量、配比、去污染。
- Packing 把短文档拼满，避免 padding 白算；平均 1024/4096 时吞吐约 4 倍。
- 预训练看 loss 曲线和 checkpoint；SFT 只在回答上算损失。
- RLHF 贵在四个模型和在线采样；DPO 用两个模型、监督式训练。
- 评测要防污染；在线指标才是最终标准。
- 失败 playbook：spike、NaN、OOM、慢数据、坏 checkpoint、标签错位。

第 13 章

分布式训练：把 7B 放进集群

一句话

单卡放不下，就把状态切开。数据并行切数据、张量并行切矩阵、流水线并行切层、ZeRO 切优化器状态。每一种切法都换来一种通信；分布式训练的本质是“用通信换显存”。

13.1 数据并行与 AllReduce

13.1.1 机制

每张卡放一份完整模型，喂不同的数据，各自算梯度；然后把所有卡的梯度求平均，各卡用同一个平均梯度更新。数学上等价于“用一个更大的 batch 训练”。

梯度同步用 AllReduce：每张卡贡献一份 S 字节的梯度，最后每张卡都拿到总和。朴素实现是参数服务器：所有卡发给中心，中心求和再广播；中心要收 $P \cdot S$，成为瓶颈。

13.1.2 Ring AllReduce

把 P 张卡围成环，分两个阶段：

- **ReduceScatter**：每轮每卡把向量的 $1/P$ 发给右邻居、从左邻居收 $1/P$，转 $P-1$ 圈后，每卡持有“某一段的全员求和”；
- **AllGather**：再转 $P-1$ 圈，把各段拼成完整结果。

每卡发送量：

$$2 \times \frac{P-1}{P} \times S \approx 2S. \tag{13.1}$$

与 P 几乎无关，这是 ring 的带宽最优性。耗时近似：

$$T \approx \frac{2S(P-1)}{PB} + 2(P-1)\alpha, \tag{13.2}$$

B 是链路带宽，α 是单跳延迟。大 S 时带宽项主导，小 S 时延迟项主导。

手算

7B 模型 bf16 梯度 14 GB，$S = 14$ GB。$P = 64$，$B = 100$ GB/s：

$$T \approx \frac{2 \times 14 \times 63}{64 \times 100} \approx 0.276 \text{ s}.$$

每步训练可能只有几百毫秒，AllReduce 占十几到几十个百分点；所以要用分层 AllReduce

（机内 NVLink、机间 IB）、梯度分桶、通信计算重叠。

13.2 ZeRO：把冗余状态削掉

数据并行每张卡都存完整优化器状态，冗余巨大。ZeRO 分三级：

阶段	切什么	每参数显存（近似）
baseline	不切	16 B + 2 B 权重
ZeRO-1	优化器状态（fp32 主权重 + m + v）	$6 + 12/P$ B
ZeRO-2	+ 梯度	$2 + (12+4)/P$ B
ZeRO-3	+ 参数	$18/P$ B

$P = 64$、7B：

$$\text{baseline} = 112\ \text{GB},\quad \text{ZeRO-1} \approx 43\ \text{GB},\quad \text{ZeRO-2} \approx 16\ \text{GB},\quad \text{ZeRO-3} \approx 2\ \text{GB}.$$

ZeRO-3 把参数也切了，前向/反向需要 AllGather 取回完整层，通信量增加；它适合“显存极度紧张、卡多”的场景。ZeRO-1/2 通信量小，更常用。

13.3 张量并行

把一层的矩阵乘切到多张卡。以 $Y = XW$ 为例：

- **列并行**：W 按列切成 $[W_1, W_2]$，每卡算 $Y_i = XW_i$，输出拼起来；前向无需通信，反向需要。
- **行并行**：W 按行切，输入 X 也按列切，每卡算部分和，最后 AllReduce 求和。

Megatron 的 Transformer 用“列并行 + 行并行”配对：QKV/MLP 第一层列并行，输出投影/MLP 第二层行并行，每层只需一次 AllReduce。

TP 的通信量约为 $B \times T \times D$ 每层，且是低延迟的 AllReduce；它必须放在 NVLink 域内（单机或超节点），跨机做 TP 通常不划算。

13.4 流水线并行

按层切：卡 1 放第 1–8 层，卡 2 放第 9–16 层……前向像工厂流水线，一张卡算完把激活传给下一张。问题是“气泡”：第一张卡在算时，后面的卡在等；最后面的卡在算时，前面的卡在等。

设 P 个流水线阶段、M 个 micro-batch，气泡比例约：

$$\text{bubble} = \frac{P-1}{M+P-1}. \tag{13.3}$$

$P = 4, M = 8$ 时气泡 $3/11 \approx 27\%$；增加 micro-batch 或使用交错调度（每个卡放多个不连续的层段）可以降低。

PP 的通信量小（只传层边界激活），适合跨机；但实现复杂，micro-batch 调度、激活显存、负载均衡都要调。

13.5 3D 并行与拓扑

把三种切法组合：

- **TP** 放在机内 NVLink（带宽最高、延迟最低）；
- **PP** 放在机间，阶段数不要太多；
- **DP** 放在剩余维度，用 ZeRO 削状态。

一个 1024 卡的例子：TP=8（单机 8 卡）、PP=8、DP=16，总卡数 $8 \times 8 \times 16 = 1024$。7B 模型每卡参数约 $6.74B/(8 \times 8) = 105M$，bf16 约 210 MB；优化器状态按 DP/ZeRO 再分。

拓扑决定性能：同一台机器内 NVLink 900 GB/s，机间 IB 50–100 GB/s，差一个数量级。把通信最重的 TP 放在机内，是 3D 并行的第一原则。

13.6 混合精度

- **fp16**：10 位尾数，精度高，但范围小（最大 65504），容易溢出；需要 loss scaling。
- **bf16**：8 位指数、7 位尾数，范围和 fp32 一样，训练稳定；精度略低，但大模型通常够用。
- **fp8**：H100 等支持，吞吐更高；需要 per-tensor scaling 和校准。

混合精度训练的典型做法：前向/反向用 bf16，主权重和优化器状态用 fp32；更新时把梯度转 fp32。这样既有吞吐，又不会把参数更新“抹平”。

13.7 激活重算

激活值占显存的大头。梯度检查点只保存每层的输入，反向时重新算中间量：

- 显存：从 $O(L)$ 降到 $O(\sqrt{L})$ 或 $O(1)$（取决于策略）；
- 计算：多一次前向，约增加 33% FLOPs。

这是“用算力换显存”的典型；在显存不够、卡又多时非常划算。

13.8 容错与 checkpoint

13.8.1 故障是日程表

1024 张卡，单卡年故障率 5%，平均每天坏：

$$1024 \times 0.05/365 \approx 0.14 \text{ 张/天},$$

平均每 7 天一次。训练跑几个月，故障不是“如果”，是“什么时候”。

13.8.2 Checkpoint 策略

- 每 N 步或每 T 分钟写一次；间隔越短，恢复损失越小，但写盘开销越大。

- 异步写：先写内存/本地盘，再后台传对象存储；避免阻塞训练。
- 原子写：临时文件 + rename；校验和；保留最近 K 份。
- 恢复：加载权重、优化器、调度器、step、数据管道位置；少一项就可能不一致。

恢复时间 = 加载 checkpoint + 回滚到上一个 checkpoint 的步数。设每 30 分钟存一次，故障平均发生在间隔中间，损失约 15 分钟训练；加载 100 GB checkpoint 可能还要几分钟。

13.8.3 弹性训练

节点故障后，用剩余节点继续，动态调整 DP 大小；需要重新分片、重建通信组。它能把“整任务重启”变成“局部恢复”，但实现复杂，通常和 ZeRO/checkpoint 配合。

13.9 MFU：卡到底有几张在干活

$$\text{MFU} = \frac{\text{实际达到的 FLOPs/s}}{\text{峰值 FLOPs/s}}. \tag{13.4}$$

7B、2T、1024 卡、1.9 天对应 50% MFU；真实大模型训练常见 35%–50%，少数优化到 55%+。损失来自：

- 通信等待（AllReduce、PP 气泡）；
- 数据加载/预处理；
- 激活重算；
- kernel 效率、显存带宽、负载不均；
- 故障重启。

MFU 是训练工程的核心 KPI，就像后台的 CPU 利用率；但它不是越高越好——为了 1% 的 MFU 花 10 倍工程成本，不一定划算。

13.10 7B 与 70B 的集群画像

	7B/2T	70B/15T
总 FLOPs	8.4×10^{22}	6.3×10^{24}
GPU（H100，50% MFU）	1024	4096
墙钟时间	约 1.9 天	约 36 天
训练显存/卡（含状态/激活）	约 60–80 GB	约 60–80 GB（靠并行切）
并行建议	TP=8, PP=1–2, DP= 余	TP=8, PP=8–16, DP= 余
主要瓶颈	通信、数据	故障、通信、稳定性

13.11 本章小结

- DP 用 AllReduce 同步梯度；ring 每卡约传 $2S$，与卡数几乎无关。

- ZeRO 逐级切优化器/梯度/参数，7B 从 112 GB 降到几十 GB 甚至几 GB。
- TP 切矩阵，通信重，放 NVLink；PP 切层，气泡 $(P-1)/(M+P-1)$，放机间。
- 3D 并行的拓扑原则：TP 机内、PP 机间、DP 剩余。
- 混合精度、激活重算、checkpoint、弹性训练是长跑的基础设施。
- MFU 是综合 KPI；7B/2T/1024 卡约 1.9 天，70B/15T/4096 卡约 36 天。

第 14 章

从单机到生产：路线图与检查清单

一句话

你不需要先成为算法专家才能做 AI Infra。后端工程师的分片、缓存、调度、容错、SLO 能力全部可迁移；需要补的是“数值直觉 + GPU 内存层次 + 训练/推理的账本”。路线图比天赋重要。

14.1 能力地图：哪些能迁移，哪些要补

旧能力	迁移到	需要补的
分片/副本/一致性	DP/TP/PP、ZeRO、AllReduce	集合通信、拓扑、带宽
缓存/内存池	KV Cache、PagedAttention、前缀缓存	显存账本、块管理
限流/队列/SLO	连续批处理、PD 分离、goodput	TTFT/TPOT、压测
容错/重试	checkpoint、弹性训练	长跑故障模型、恢复语义
容量规划	算力/显存账本、MFU	$6ND$、roofline、GPU 规格
可观测性	loss 曲线、梯度范数、GPU 指标	训练诊断、loss spike

需要从零补的数学不多：向量/矩阵、导数/链式法则、softmax/交叉熵、浮点数。本书第 2 章就是最小集合；剩下的可以在项目里现学。

14.2 30/60/90 天计划

14.2.1 第 1 个月：建立直觉

- 读完本书第 1–7 章，跑通 Python/Go demo；
- 手推一遍 2 参数神经元、softmax+CE 梯度、$6ND$；
- 用 `nvidia-smi`/`py-spy`/`nsys` 看一个真实推理服务；
- 写一个 benchmark：固定模型，测 batch 1/8/32 的 TTFT、TPOT、吞吐。

14.2.2 第 2 个月：动手实现

- 给 demo 加连续批处理：请求队列、KV 块池、抢占；

- 实现 int8 量化 + 一个简单的 paged KV allocator；
- 用 PyTorch 训练一个 10M–100M 参数的小模型（单卡/单机）；
- 读 vLLM/llama.cpp 的调度与 KV 管理代码，画一张调用图。

14.2.3 第 3 个月：工程化

- 把一个开源 7B 模型用 vLLM 部署，做压力测试和容量规划；
- 用 DeepSpeed/FSDP 跑一次多卡微调，记录 MFU、显存、通信；
- 做一次故障演练：kill 一个 worker，验证 checkpoint 恢复；
- 写一份推理服务的 SLO/容量/降级方案。

14.3 三个练手项目

14.3.1 项目 A：给微型推理引擎加 batching

在本书 `py/infer.py` 基础上：

1. 请求队列，每个请求有 prompt、max_tokens、已生成 token；
2. 每步把活跃请求拼成一个 batch（先不做 paged，按最大长度预留）；
3. 实现连续批处理：完成的请求退出，新请求插入；
4. 统计 slot 利用率、TTFT、TPOT，和 `py/schedule.py` 的模拟对比。

这个项目能把第 7、8 章的“调度”从概念变成代码。

14.3.2 项目 B：PagedAttention 模拟器

不用 GPU，纯 Python/Go 实现：

1. 固定大小的 KV 块池（如 16 token/块）；
2. 每个请求一张 block table；
3. 分配、释放、前缀共享（引用计数）；
4. 对比“按最大长度预留”和“按需分页”的显存利用率。

它不涉及 kernel，但能把“虚拟内存”讲清楚；面试时能画图，比背概念有用。

14.3.3 项目 C：单卡训练 100M 模型

用 PyTorch + 一个小语料（如 TinyStories）训练 100M 参数模型：

1. 数据：tokenize、packing、train/val 切分；
2. 训练：bf16、AdamW、warmup/cosine、梯度裁剪；

3. 监控：loss、grad norm、MFU、显存；
4. 推理：KV Cache、采样、和本书 demo 对比。

它能把本书的 6472 参数经验放大到“真实但可负担”的尺度。

14.4 自测题

1. 为什么 Prefill 是计算密集、Decode 是访存密集？用算术强度解释。
2. KV Cache 每 token 多少字节？7B/GQA/4k 上下文是多少？
3. 为什么 softmax+CE 的梯度是 $p - \text{one-hot}$？
4. 反向传播为什么约是前向的 2 倍？
5. 7B/2T 的 $6ND$ 是多少？1024 张 H100、50% MFU 要跑多久？
6. ZeRO-1/2/3 各切什么？7B、64 卡各剩多少显存？
7. TP 为什么放机内、PP 为什么放机间？
8. 连续批处理为什么能提高 slot 利用率？它的代价是什么？
9. PagedAttention 解决了什么浪费？块大小如何取舍？
10. DPO 为什么比 PPO 便宜？代价是什么？

14.5 常见误区

常见坑

- **“参数量越大越好。”** 数据、训练 token、推理成本同样重要；小模型 + 好数据常常打败大模型 + 脏数据。
- **“GPU 利用率高就是快。”** 可能是无效计算（padding、重算）；要看 goodput 和 MFU。
- **“量化一定掉点。”** W8A16 往往几乎无损；要做任务级评估，不只看困惑度。
- **“batch 越大吞吐越高。”** 超过显存/SLO 后，TTFT/TPOT 崩，goodput 反而下降。
- **“训练完就结束了。”** 推理成本可能远超训练；部署、监控、迭代才是长跑。
- **“先学完理论再动手。”** 这本书的路线是“跑通—手算—工程”，反过来更容易放弃。

14.6 继续阅读

- 原书《大模型训练与推理工程详解》：系统架构、集群、成本、转型路线。
- Attention Is All You Need：Transformer 原始论文。
- GPT-3、Chinchilla、LLaMA：规模、数据、架构的演进。

- vLLM、PagedAttention、FlashAttention、ZeRO、Megatron-LM：工程系统。
- InstructGPT、DPO：对齐方法。
- llama.cpp、vLLM、SGLang：推理引擎源码。

14.7 最后

大模型不是魔法，它是一台巨大的、用梯度调参数的函数拟合机器。你已经会写高并发后端，说明你能理解状态、缓存、队列、失败和成本；剩下的只是把这套直觉换到“显存、带宽、token”这三个坐标轴上。

先跑通那个 6472 参数的小模型。当你能指着日志里的每一个数字说清楚它是哪一步算出来的，你就已经越过大多数“只会调包”的人了。

第 A 章

数学推导补充

A.1 Softmax 的 Jacobian

设 $p_i = e^{z_i}/\sum_j e^{z_j}$。当 $i = j$：

$$\frac{\partial p_i}{\partial z_i} = \frac{e^{z_i}\sum_j e^{z_j} - e^{z_i}e^{z_i}}{(\sum_j e^{z_j})^2} = p_i(1 - p_i).$$

当 $i \neq j$：

$$\frac{\partial p_i}{\partial z_j} = \frac{0 - e^{z_i}e^{z_j}}{(\sum_k e^{z_k})^2} = -p_i p_j.$$

合并：

$$\frac{\partial p_i}{\partial z_j} = p_i(\mathbf{1}[i = j] - p_j).$$

A.2 交叉熵对 logits 的梯度

$L = -\ln p_y$：

$$\frac{\partial L}{\partial z_j} = -\frac{1}{p_y}\frac{\partial p_y}{\partial z_j} = -\frac{1}{p_y}p_y(\mathbf{1}[y = j] - p_j) = p_j - \mathbf{1}[y = j].$$

对 T 个位置取平均，再除以 T。

A.3 LayerNorm 反向

记 $\mu = \frac{1}{D}\sum x_i$，$\sigma^2 = \frac{1}{D}\sum(x_i - \mu)^2$，$\hat{x}_i = (x_i - \mu)/\sigma$，$y_i = \gamma_i\hat{x}_i + \beta_i$。

给定 dy_i：

$$d\hat{x}_i = dy_i\gamma_i, \qquad d\gamma_i = dy_i\hat{x}_i, \qquad d\beta_i = dy_i.$$

对 x_i：

$$dx_i = \frac{1}{D\sigma}\left(D\,d\hat{x}_i - \sum_j d\hat{x}_j - \hat{x}_i\sum_j d\hat{x}_j\hat{x}_j\right).$$

推导要点：μ 和 σ 都依赖所有 x_i，所以每个 x_i 的梯度要收集整行的 $d\hat{x}$ 和 $d\hat{x}\hat{x}$。

A.4 注意力反向

前向：

$$S = \frac{QK^\top}{\sqrt{D}} + M, \quad A = \mathrm{softmax}(S), \quad O = AV.$$

给定 dO：

$$dV = A^\top dO, \qquad dA = dO\, V^\top.$$

softmax 反向：

$$dS = A \odot \left(dA - \left(\sum_j A_{ij} dA_{ij} \right) \mathbf{1}^\top \right).$$

然后：

$$dQ = \frac{1}{\sqrt{D}} dS\, K, \qquad dK = \frac{1}{\sqrt{D}} dS^\top Q.$$

掩码位置 $dS_{ij} = 0$。

A.5 AdamW 的偏差修正

m_t 是梯度的指数滑动平均，初始为 0，所以早期偏小。其期望：

$$\mathbb{E}[m_t] = (1-\beta_1) \sum_{i=1}^{t} \beta_1^{t-i} \mathbb{E}[g_i] \approx (1-\beta_1^t)\mathbb{E}[g].$$

因此除以 $1-\beta_1^t$ 做无偏修正；二阶矩同理除以 $1-\beta_2^t$。

A.6 $6ND$ 的矩阵乘推导

设一层线性 $Y = XW$，$X \in \mathbb{R}^{T\times D}$，$W \in \mathbb{R}^{D\times D'}$。前向 FLOPs：

$$2TDD'.$$

若 W 有 DD' 个参数，则每个 token 约 $2DD'$ FLOPs，即 “每参数 2 FLOPs”。所有层加起来约 $2N$ 每 token。

反向对 W 的梯度 $X^\top dY$：$2TDD'$；对 X 的梯度 $dYW^\top$：$2TDD'$。合计 $4TDD'$，即每 token $4N$。所以前向 + 反向 $= 6N$ 每 token。

第 B 章

术语表

术语	含义
Token	模型处理的最小单位；一个字符、子词或词
Tokenizer	文本 $\leftrightarrow$ token ID 的转换器
Embedding	token ID 到向量的查表矩阵
Logits	模型最后一层的未归一化分数
Softmax	把 logits 变成概率分布
Cross-Entropy	$-\ln p_y$，分类/下一词预测的损失
Perplexity	e^L，模型平均在多少个 token 里犹豫
Prefill	对 prompt 的一次并行前向
Decode	逐 token 的串行生成
KV Cache	缓存历史 token 的 K/V，避免重算
TTFT	Time To First Token，首 token 延迟
TPOT	Time Per Output Token，相邻 token 间隔
Goodput	满足 SLO 的有效吞吐
Arithmetic Intensity	FLOPs / Bytes，判断计算/访存密集
Roofline	算力/带宽拐点的性能模型
MFU	Model FLOPs Utilization，算力利用率
AllReduce	所有卡归约并广播结果
Ring AllReduce	环形两阶段 AllReduce，带宽最优
ZeRO	分片优化器/梯度/参数的显存优化
TP	Tensor Parallel，张量并行
PP	Pipeline Parallel，流水线并行
DP	Data Parallel，数据并行
PagedAttention	用分页管理 KV Cache
Continuous Batching	iteration 级动态批处理
Speculative Decoding	小模型起草、大模型验收
PD Separation	Prefill/Decode 分离部署
SFT	Supervised Fine-Tuning，监督微调
RLHF	基于人类反馈的强化学习
DPO	Direct Preference Optimization
LoRA	低秩适配器，参数高效微调
Checkpoint	训练状态快照

Activation Checkpointing		用重算换激活显存

第 C 章

代码地图与运行命令

C.1 目录

```
llmbook/
  data/corpus.txt
  py/autograd.py model.py train.py infer.py neuron.py
     bpe.py sft_dpo.py schedule.py quant.py
  go/go.mod tiny/ cmd/{infer,train,neuron}/
  weights/ out/ figures/
  run_all.sh
```

C.2 命令

```
cd llmbook
./run_all.sh

# 或者分步
python3 py/neuron.py
cd go && go run ./cmd/neuron

python3 py/train.py --steps 600 --batch 4
cd go && go run ./cmd/train -steps 1200

python3 py/infer.py --temperature 0
cd go && go run ./cmd/infer -weights ../weights/tiny_py.json -temp 0
```

C.3 预期结果

命令	关键输出	说明
`py/neuron.py`	loss $4 \to 3.8 \times 10^{-6}$	10 步收敛
`py/train.py`	loss 0.208，样本 `the cat ran...`	600 步，约 100 s
Go train	loss 0.212	1200 步，约 2 s
Python infer	`the cat ran to the log .`	贪心
Go infer	同上	与 Python 逐 token 一致
`py/quant.py`	top-1 MATCH/DIFF	量化误差演示
`py/schedule.py`	64.5% vs 94.2%	静态/连续 batch

第 D 章

练习答案（选）

第 6 章

1. bf16：$2 \times 1 \times 24 \times 2 = 96$ B/token；23 token 共 2208 B。
2. int8：$2 \times 32 \times 8 \times 128 \times 1 = 65536$ B=64 KiB/token；4k 上下文 64 KiB $\times$ 4096 = 256 MiB。
3. 权重读取量不变（14 GB 只读一遍）；KV 读取量随 batch 线性增长；算术强度近似随 batch 增长，直到 KV 读取占主导。
4. 不追加 K/V，缓存长度不增长，注意力只能看到旧 token；输出会乱，对拍差值非 0。

第 8 章

1. 连续批处理把完成的请求立刻移出、新请求立刻插入，slot 不空转；代价是调度复杂、TTFT/T-POT 互相影响、需要抢占。
2. PagedAttention 解决“按最大长度预留 + 外部碎片”；块越小碎片越少但 block table 和寻址开销越大，16 是经验值。
3. 量化省显存/带宽，但误差会改变近邻 token 的排序；要用任务指标评估，不只看困惑度。
4. PD 分离让 Prefill/Decode 独立扩缩容、互不干扰；代价是 KV 传输、故障恢复、两池配比。

第 11 章

1. $6ND = 6 \times 7 \times 10^9 \times 2 \times 10^{12} = 8.4 \times 10^{22}$；$1024 \times 5 \times 10^{14} = 5.12 \times 10^{17}$ FLOPS，时间 1.64×10^5 s≈ 1.9 天。
2. 7B/64 卡：baseline 112 GB；ZeRO-1 约 43 GB；ZeRO-2 约 16 GB；ZeRO-3 约 2 GB（不含激活）。
3. TP 通信最重、延迟最敏感，放 NVLink；PP 通信量小，放机间；DP 放剩余维度。

第E章

参考资料

1. Vaswani et al. *Attention Is All You Need*. NeurIPS 2017.
2. Brown et al. *Language Models are Few-Shot Learners*. NeurIPS 2020.
3. Hoffmann et al. *Training Compute-Optimal Large Language Models*. NeurIPS 2022.
4. Touvron et al. *LLaMA: Open and Efficient Foundation Language Models*. 2023.
5. Kwon et al. *Efficient Memory Management for Large Language Model Serving with PagedAttention*. SOSP 2023.
6. Dao et al. *FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness*. NeurIPS 2022.
7. Rajbhandari et al. *ZeRO: Memory Optimizations Toward Training Trillion Parameter Models*. SC 2020.
8. Shoeybi et al. *Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism*. 2019.
9. Ouyang et al. *Training Language Models to Follow Instructions with Human Feedback*. NeurIPS 2022.
10. Rafailov et al. *Direct Preference Optimization: Your Language Model is Secretly a Reward Model*. NeurIPS 2023.
11. 大模型训练与推理工程详解. 2026.